The Bonded Commons:

Pre-Transactional Trust Infrastructure for Multi-Agent Systems

Erich Owens
Curiositech LLC
engineering@portdaddy.dev

with the competitive-insurance pricing mechanism (§7.5.8) by
Thomas Youle
Business Economics & Public Policy
Indiana University

September 2026
Version 2.8 (textbook edition)

Abstract

Two autonomous agents working on the same project corrupt each other's state, double-claim the same files, race for the same ports, and leave the operator to do recovery archaeology. The problem is not the agents—it is that they have no coordination substrate. This paper specifies one. We model the daemon the way game theory models a referee who never forces anyone's hand: it whispers a private recommendation to each agent—*you go, you wait*—and Aumann's theory of *correlated equilibrium* [5] tells us exactly when agents have no reason to ignore it. That "exactly when" is the decisive part: the recommendations constitute a correlated equilibrium only when the explicit obedience inequalities of §2.1 hold. The substrate has three independently necessary layers—capability-attenuated tokens that bound scope, Merkle-chained evidence trails that preserve attribution, and collateralized work contracts that pre-fund damage regardless of intent. We formalize the construction in the context of Port Daddy, a coordination daemon for local agent swarms, and separate structural guarantees from incentive assumptions. Sen's Impossibility of a Paretian Liberal supplies a design warning, not a proof that every enforced lock is inefficient: decisive local control and team-wide Pareto choice can conflict under unrestricted preferences. That warning motivates an advisory claim graph whose completeness is proved separately. We further specify a *mutable-signal ledger* (§4.3) supporting revocation, renaming, and provenance-aware propagation without sacrificing attribution integrity. Finally, we evaluate a competitive-insurance pricing mechanism (§7.5.8, contributed by Youle) against a named static baseline. Its Pareto comparisons are simulation results under stated assumptions, not a universal welfare theorem. Hobbes had the right idea about sovereigns; he just lacked a way to deploy one on `localhost:9876`.

Keywords: multi-agent coordination, trust infrastructure, capability-based security, social contract, collateralized computation, formal verification, commons governance

Reading time: approximately 50 minutes end-to-end. The Reader's Map below provides shorter paths for specific reader types.

PORT DADDY COORDINATION PAPERS **CHAPTER 7 OF 8**

Part I Ground Truth	1. The Single-Writer Kernel · 2. The Anchor Protocol · 3. The Sealed Harbor
Part II The Cost of Seeing	4. The Legible Swarm
Part III What Survives the Restart	5. From Spawn to Person
Part IV Trade Between Strangers	6. The Harbor Economy · **7. The Bonded Commons** (this chapter) · 8. The Federated Harbor

The Harbor, the Person, and the Economy reads in this order: each chapter stands on the ones before it, and each proof chapter follows the chapter whose promises it keeps. Every chapter is independently readable.

Reader's Map

The paper builds in four layers; each layer carries a critical formal claim or empirical result. If you have less than 50 minutes, the table below tells you where to start.

Layer	Critical artifact	Lives at
(1) Structural prevention	Capability-attenuated tokens; ProVerif proofs of algorithm-confusion immunity and delegation replay resistance	§3–§3.2; *The Anchor Protocol*
(2) Immutable attribution	Merkle Forest with cross-daemon inclusion proofs; EasyCrypt binding mechanization (`binding.ec`)	§4.2; Appendix A
(3) Economic alignment	Competitive-insurance auction; conditional Monte Carlo comparison	§7.5.8; §7.5.8
(4) Governance	Sen-inspired design warning; advisory-conflict completeness	§5–§5.2

Suggested reading paths by reader type:

- **Formal-methods reviewer / program committee** — abstract, then jump to Appendix A (Formal Verification Status). The artifact registry is the most concentrated signal of what is and is not proved.
- **Engineering manager evaluating multi-agent infrastructure** — abstract, §2.2 ("Why Agents Consent"), §7 (bond pricing), §7.5.8 (welfare results), then Appendix B (one $5 task traced end to end). The Appendix A status registry tells you what is production-ready and what is research.
- **Cryptoeconomic protocol designer** — abstract, then §7.5.8 (auction mechanism), §7.5.8 (welfare theorem with inline Pareto Monte Carlo), §7.5.9 (A5 Sybil-deterrence is coverage-bounded — a surprising negative result).

- **AI safety researcher** — §6.1 ("What Morality Means When Death Is Cheap"), §5 (Sen's theorem), §7 (collateralized work contracts). Skim Appendix A for the proof posture.
- **Distributed systems engineer** — §3–§4.2 (primitives), then *The Anchor Protocol* for the cryptographic-token protocol detail.
- **Policy / governance analyst** — abstract, §5–§5.2 (the Sen's-theorem argument for advisory rather than enforced claims), §12 (federated sovereign).

Volume Context. *The Anchor Protocol* supplies the ProVerif-verified cryptographic-token foundation this chapter assumes throughout—Harbor Card phases, cuckoo-filter revocation, bounded multi-hop delegation; this chapter supplies the economic and governance argument built on top of it.

Contents

[1]This subsection contributed by Thomas Youle (Indiana University, Business Economics & Public Policy) based on conversations with the primary author in March–April 2026. The mechanism builds on classical competitive-insurance market literature [21, 22] adapted to the agent-transactions setting. The text here is a pre-print draft pending economist review of §7.5.8–§7.5.8.

1 Introduction: The Trust Problem

Two autonomous coding agents share a repository. Agent A is asked to refactor the authentication middleware. Agent B, spawned ninety seconds later from a sibling worktree, is asked to add a rate-limiter to the same middleware. Neither knows the other exists. They both edit, both commit, both push; one of them gets a merge conflict it cannot interpret, retries with a hallucinated resolution, and corrupts the file. The operator returns from a coffee break to a working tree she does not recognize. This is not a hypothetical—it is the modal failure mode of every multi-agent setup we have inhabited. Existing frameworks do not address it: LangGraph, CrewAI, and AutoGen handle state graphs and role assignment competently; the Model Context Protocol standardizes agent-tool integration. None of them give Agent A and Agent B a way to know about each other. The problem is **trust**—specifically, trust as infrastructure rather than trust as per-transaction ceremony.

When Agent A delegates a subtask to Agent B, it implicitly trusts that B will not corrupt the shared state, will not exceed the scope of the delegation, and will not silently fail in a way that poisons A's downstream work. When Agent B accepts the delegation, it implicitly trusts that A's description of the task is accurate, that the resources A promised are available, and that A will still exist when B finishes. These implicit trust assumptions pervade every multi-agent interaction, and every one of them can be violated.

The standard response in security engineering is "zero trust"—never trust, always verify. But zero trust applied literally to agent coordination is paralytic. If every message requires cryptographic verification, every delegation requires capability negotiation, and every file access requires authorization—the coordination overhead overwhelms the work itself. Agents spend their context windows negotiating trust instead of solving problems.

Hobbes identified this dynamic in 1651 [1]. In the state of nature, without a common authority, rational actors cannot cooperate because the cost of verifying each counterparty's intentions exceeds the benefit of cooperation. The solution is not to make individuals trustworthy—it is to establish a **sovereign** whose authority both parties accept *before* the interaction begins, so that the interaction itself can focus on the work. "Covenants, without the sword, are but words, and of no strength to secure a man at all."

This paper argues that multi-agent systems need exactly this: a **commons authority** that provides trust as infrastructure, not trust as ceremony. Agents don't negotiate trust per-transaction. They enter a commons where trust has already been established—structurally, evidentially, and economically—and they work freely within that trust envelope.

1.1 The Three Failures of Peer-to-Peer Trust

Peer-to-peer trust fails for agent systems in three specific ways:

It doesn't scale. If n agents must establish pairwise trust, the system requires $O(n^2)$ trust negotiations. Each negotiation consumes tokens, time, and context window. With 8 agents, this is manageable. With 50, it dominates wall-clock time. With 1000 (a production agent fleet), it is infeasible.

It doesn't survive death. When an agent crashes, its trust relationships die with it. A successor agent that resumes the dead agent's work must re-establish trust with every counterparty from scratch. The trust investment is non-transferable because it was encoded in ephemeral bilateral state, not in durable infrastructure.

It doesn't address misaligned principals. An agent can be perfectly "trustworthy" in the sense that it faithfully executes its instructions—while its instructions come from a principal whose interests are adversarial to the commons. Peer-to-peer trust evaluates the agent's behavior;

it cannot evaluate the principal's intent. The agent is a loyal soldier in someone else's war, with a spotless reputation.

1.2 Contributions

We present:

1. A reading of the coordination daemon as a **candidate correlating device** in the sense of Aumann [5]: claim recommendations are private signals drawn from a common-knowledge distribution, while incentive compatibility remains conditional on explicit obedience inequalities (Section 2, esp. §2.1).
2. A **three-layer trust architecture** (structural, evidentiary, economic) that does not depend on agent morality, shared values, or threat of termination (Sections 3–7).
3. A Sen-grounded **design warning** about decisive resource rights under private information, plus a separate completeness theorem for **advisory resource claims** (Section 5).
4. A **TLA$^+$ specification** of the coordination lifecycle with crash recovery, verifying safety and liveness properties (Section 10).
5. The design of a **collateralized work contract** system (Float Plans) that prices agent risk and settles outcomes mechanically, transforming the moral question of agent trustworthiness into the economic question of adequate bonding (Section 7).

The security layer (agent authentication and capability delegation) is provided by the Anchor Protocol [11], an accompanying chapter. This paper builds the trust, governance, and economic layers on top of that cryptographic foundation.

1.3 Related Work in Crypto-Economic Bonding

The architecture sits at the intersection of four bodies of prior work, each of which we extend rather than replicate.

Bonded participation in distributed systems. Proof-of-stake consensus protocols (Casper [29], Tendermint [30], and the Ethereum 2.0 specification [31]) introduced *slashing*: validators post collateral that is forfeited on equivocation or liveness failure. The technique is the closest analogue to our bonded-commons settlement, and the proofs of soundness (a rational validator strictly prefers honest behavior when slash $>$ deviation gain) are direct ancestors of §7. We depart from this lineage on three axes: (i) the adversary is co-located on a single machine, not a Byzantine network, so consensus reduces to a daemon write rather than a global commit; (ii) the bonded act is *coordination*, not validation—work that produces side effects in a shared file system, not signatures on blocks; and (iii) the principal is human-or-LLM mixed, so the mechanism must price two failure modes (malice and confusion) the blockchain lineage collapses into one.

Capability-based security. Dennis and Van Horn [15], Miller's object-capability work [32], and the SPKI/SDSI line [33] provide the structural attenuation that we lift wholesale into the Anchor Protocol layer. Our contribution is composing capability attenuation with bonding: a capability without economic settlement still permits the holder to "correctly" execute a destructive action; bonding without capability bounding still permits unbounded blast radius before slash. Both layers are necessary.

Commons governance. Ostrom's eight design principles for governing common-pool resources [28] predict that durable cooperation emerges only when boundaries, monitoring, graduated sanctions, and dispute resolution are co-located with the commons. The bonded commons satisfies these requirements as services of the daemon rather than as social institutions, and §5.3 renders the dispute-resolution and appeals path explicitly.

Cryptoeconomic mechanism design. Werbach [34], Buterin [35], and the broader DAO literature establish "code is law" as a posture toward enforcement. Our position is narrower and weaker on purpose: code is the *record*, not the law. Allocation decisions remain with the agents (Sen, §5); the daemon supplies the evidence and the settlement. This deliberate weakening is what makes coercive enforcement avoidable while keeping accountability intact.

Table 1 compresses the four paragraphs above into what is borrowed, what is departed from, and why.

Table 1: Every borrowed result is re-used at a smaller scale than its origin assumed: the adversary is co-located rather than networked, the bonded act is coordination rather than validation, and the record is evidence rather than law. Nothing in the four lineages is discarded — each is narrowed by one assumption, and the narrowing is what makes the composition deployable on one machine.

Lineage	What we borrow	What we depart from	Why
Bonded participation (PoS slashing)	Slash->-deviation-gain soundness argument	Byzantine networked adversary; validation as the bonded act	Adversary is co-located on one machine, so consensus reduces to a daemon write, not a global commit
Capability-based security	Offline attenuation; authority only ever narrows	Capabilities alone as the whole story	A correctly-scoped token still permits a destructive in-scope write; bonding prices what the wall admits
Commons governance (Ostrom)	Boundaries, monitoring, graduated sanctions, dispute resolution	Social institutions as the enforcement medium	The four functions ship as daemon services (§5.3), not as norms an agent must internalize
Cryptoeconomic mechanism design	Priced deterrence; market-discovered parameters	"Code is law" as an enforcement posture	Code is the *record*, not the law: allocation stays with agents (§5); the daemon supplies evidence and settlement

What is novel. The synthesis: bonded participation + capability attenuation + advisory (not enforced) coordination + immutable attribution, packaged as localhost infrastructure rather than a global protocol. We are not aware of a prior system that combines all four in this form; each component has precedent, while the composition targets auditable coordination and attribution that survives identity disposal. Its welfare effects remain conditional on the explicit payoff and monitoring models developed below. Table 2 shows the composition.

Table 2: Three layers answer three different questions about the same attempted act. An out-of-grant act ends at Layer 1 before shared state changes; every admitted act crosses Layer 2 and acquires durable attribution, so admission creates an attribution duty; a harmful admitted act reaches Layer 3, where only bound evidence can reach settlement. [internal]

Layer	Question	What it does to the act	Terminal outcome
1 scope	can this act exist?	terminates an out-of-grant trajectory before shared state changes	refused
2 evidence	who did what?	binds every admitted act to a durable, addressable attribution	complete and recorded
3 collateral	who carries the loss?	prices a harmful admitted act against posted collateral; only bound evidence can reach settlement	priced settlement

2 The Commons Authority

Return to Agent A and Agent B from the introduction's opening scene—the two agents who corrupted the same middleware because neither knew the other existed. Nothing about that failure required malice, and nothing about it would have been fixed by making either agent more careful. What was missing was a third party that both agents were already talking to: something that knew A had the file, could tell B so at claim time, and would still remember the whole episode after both processes exited.

That third party has to be settled *before* any interaction, not improvised transaction by transaction—otherwise establishing it is just the $O(n^2)$ trust negotiation of §1.1 under a new name. We call it the *commons authority*: one always-on service that every agent already trusts, precisely because it was there before any of them showed up. Five capabilities make it that, and the definition below names them.

Definition 2.1 (Commons Authority)**.** A commons authority $\mathcal{D}$ for a multi-agent system is a persistent service that provides:

1. **Identity**: Every agent receives a cryptographic identity before participating.
2. **Capability bounding**: Every agent's operations are structurally limited by attenuable tokens.
3. **Shared knowledge**: Resource claims, agent status, and conflict information are visible to all participants.
4. **Immutable history**: All actions are recorded in an append-only, tamper-evident trail.
5. **Economic settlement**: Work contracts are collateralized and settled against verifiable evidence.

The commons authority is not a central planner. It does not decide which agent should work on which task, which file conflicts should be resolved in whose favor, or which agents are "good." It provides the *infrastructure* within which agents make those decisions for themselves, using the shared knowledge and economic incentives the authority maintains.

Remark. The commons authority is a *Hobbesian sovereign*, not an *omniscient coordinator*. Hobbes's Leviathan does not micromanage citizens' daily choices—it establishes the conditions (laws, enforcement, courts) under which citizens can trust each other enough to transact freely. Similarly, the commons authority establishes identity, capability bounds, shared knowledge, and economic settlement—and then gets out of the way.

In the Port Daddy implementation, the commons authority is a daemon running on `localhost:9876`, backed by SQLite. The simplicity of the implementation is deliberate: the authority's value comes from its *guarantees*, not its complexity.

2.1 The Daemon as Correlating Device

Picture a crossing guard rather than a traffic light. A traffic light gives everyone the same instruction on a fixed cycle regardless of what is actually happening at the intersection; a crossing guard watches the specific cars and pedestrians in front of her and privately waves one through while holding another back. The daemon plays crossing guard, not traffic light: it sees the whole intersection (the claim graph) and issues one private recommendation per agent. But a crossing guard has no legal authority over anyone—drivers obey because obeying is better for them than the alternative, not because they must. Aumann's test for whether such private recommendations actually change behavior, rather than being safely ignored, is the obedience condition below.

The daemon's role therefore admits a precise game-theoretic test. Aumann [5] introduced the *correlated equilibrium*: a mediator draws a recommendation tuple $r = (r_1, \ldots, r_n)$ from a joint distribution μ over action profiles and privately reveals r_i to player i. The distribution is a correlated equilibrium only if, for every player i, recommended action r_i, and unilateral deviation a_i', the obedience inequality holds:

$$\sum_{r_{-i}} \mu(r_i, r_{-i}) \left[u_i(r_i, r_{-i}) - u_i(a_i', r_{-i}) \right] \geq 0.$$

Every mixed Nash equilibrium induces such a distribution, but a mediator does not create an equilibrium merely by issuing recommendations. The inequality is the governing condition.

The commons authority can play this mediator role. It observes the global claim graph and emits a recommendation pair on a contested file: *Agent α, proceed; Agent β, defer or coordinate.* A deterministic policy can still induce a non-degenerate μ through uncertainty over observed system states, but determinism is not itself evidence of obedience. Utilities, bond coverage, and continuation values must make the inequality above hold.

Property 2.1 (Conditional Correlating-Device Interpretation)**.** Port Daddy's daemon supplies the information structure of a correlating device for the agent-coordination game. It induces a correlated equilibrium only under the following conditions:

1. **Common-knowledge distribution.** The daemon's recommendation policy—advise-claim, defer, back-off, settle-now, halt-on-conflict—is public, deterministic given the conflict graph, and reproducible from the evidence trail of §4. Agents do not need to trust the daemon's motives; they need only verify the policy and observe its inputs.
2. **Private signal per agent.** On a contested claim, each agent receives only its own recommendation. Other agents' recommendations are observed indirectly through the public conflict graph and the note trail, never read off the wire.
3. **Obedience inequality.** For every recommendation and deviation, the deviation gain is no larger than the expected bond loss plus discounted reputation loss. Conflict-graph completeness makes a deviation observable; it does not by itself make the deviation unprofitable.

The reframe therefore yields an audit obligation, not an automatic guarantee. Nothing physically stops an agent from ignoring advice. The repeated-game calculation in Section 7 verifies one stylized two-player payoff table; deployments with different utilities must re-check the obedience inequalities. The TLA$^+$ model in Section 10 checks lifecycle safety and liveness, not strategic optimality.

No price-of-anarchy bound follows from this interpretation alone. The simulations of §7.5.8 compare one auction model with one static baseline; an analytical worst-equilibrium bound for the coordination game remains open.

Formal-backing status of the game-theoretic core. One disclosure belongs here, at the point the game theory enters, rather than buried in the appendix: in the maturity vocabulary of Appendix A, this chapter's game theory is *spec-only at the level of the research program*,

whatever each individual artifact's status in Table 9. This chapter's game-theoretic core—the correlating-device reading above, the Sen-inspired design warning (§5), the claim-signaling incentive-compatibility argument (§7.4), the competitive-insurance mechanism and its welfare comparison (§7.5.8), and the cartel folk-theorem analysis (§7.5.10)—has *no companion formal paper*. The structural and cryptographic half of this chapter is backed by *The Anchor Protocol* [11], which carries the ProVerif models and the attenuation proofs; the economic half is argued here, from scratch, and exists nowhere else in the program. What supports it is exactly what Table 9 lists—mechanized checks and Monte Carlo sweeps over supplied models, several of them PARTIAL by that table's own grading—and not a peer-reviewable formal derivation a reader can be pointed to. Read this material at that maturity level.

2.2 Why Agents Consent

In Hobbes, consent to the sovereign is rational because the alternative—the state of nature—is worse. For AI agents, consent to the commons authority is rational because agents without it are strictly worse off:

Table 3: The commons authority converts six failure modes from ad hoc and manual to deterministic and automatic — without forbidding anything an agent could already do. Every row is a service the agent gains, not a permission it loses, which is why consent is rational rather than coerced.

Capability	Without Authority	With Authority
Port assignment	Race condition	Deterministic, collision-free
File coordination	Discover conflicts at merge	Detect conflicts at claim time
Crash recovery	Work lost permanently	Successor reads immutable trail
Delegation	Bilateral trust negotiation	Attenuated capability tokens
Reputation	Every interaction from zero	History persists across lifetimes
Accountability	Anonymous harm	Full attribution via evidence chain

The daemon does not need to threaten agents. It provides services that make coordination *rational*. Agents that bypass the daemon are not punished—they are simply worse off.

3 Layer 1: Structural Prevention

The first layer of trust does not depend on agent reputation or economic incentives. It depends on *walls*: structural authorization checks that reject out-of-scope operations wherever the runtime actually interposes them.

3.1 Capability Attenuation

The Anchor Protocol [11] provides Harbor Cards—Ed25519-signed capability tokens that bound the operations available to each agent. The critical property is **offline attenuation**: when Agent A delegates to Agent B, it creates a new token whose capabilities are a strict subset of its own. Agent B can further delegate to Agent C with even narrower capabilities. Capabilities can only shrink along the delegation chain, never grow.

The capability subset check and the secret-independent comparison are implemented in a **Rust core** (`harbor-card-rs`) that is compiled to a shared library and called from the Node.js daemon via FFI. Kani harnesses check that the parser and the comparator cannot panic on bounded symbolic inputs and exercise the subset check on two concrete vectors; the every-hop attenuation property is ProVerif's, in *The Anchor Protocol*; runtime conformance tests cover the bridge. Compiler behavior, full interception coverage, and hardware side channels remain separate obligations.

The guarantee below is what security engineers mean by a *wall* rather than a *law*: not an instruction an agent could disobey, but a bound on which operations are even representable.

Definition 3.1 (Capability-Bounded Transaction)**.** An agent transaction $\mathcal{T}_\alpha$ is capability-bounded by token τ_α if every operation o executed during $\mathcal{T}_\alpha$ satisfies $o \in C(\tau_\alpha)$, where $C(\tau)$ extracts the capability set from a Harbor Card. The commons authority rejects any operation $o \notin C(\tau_\alpha)$ at the verification step, before the operation can affect shared state.

Theorem 3.1 Structural Scope Limitation. For any delegation chain $\tau_0 \to \tau_1 \to \cdots \to \tau_k$, where τ_0 is issued by the commons authority and each τ_{i+1} is attenuated from τ_i:

$$C(\tau_k) \subseteq C(\tau_{k-1}) \subseteq \cdots \subseteq C(\tau_0).$$

Within an execution that routes every relevant operation through the verifier, no accepted operation can lie outside the scope originally granted by the authority.

Proof. The Anchor model establishes a non-injective authentication correspondence for accepted modeled tokens and separately checks attenuation at the modeled chain depth [11]. Given those model assumptions and the theorem's premise that every operation passes through the verifier, each accepted hop satisfies the subset relation; transitivity yields the displayed chain. The correspondence alone does not prove replay freedom, arbitrary-depth delegation, or complete runtime interception, which are separate obligations. □

Remark. This is the layer where the metaphor of "walls, not laws" applies. A law says "you shall not write to the production database." A wall means your token is not valid for production database writes, and no amount of intent—good or bad—can change that. Laws require enforcement; walls require only construction.

Exercise 7.1, page 45.

3.2 Isolation Domains

Beyond capability tokens, the commons authority supports **structural isolation** via git worktrees: physically separate copies of the repository where agents operate on disjoint filesystem state.

Definition 3.2 (Isolation Levels)**.** Three tiers of isolation are available, ordered by strength:

I_0 **(None):** Agents share a working directory. No conflict detection. Maximum parallelism, no safety.

I_1 **(Advisory):** Agents share a working directory but register file claims with the commons authority. Conflicts are detected and reported to all participants. Claims are not enforced. This is the default.

I_2 **(Structural):** Each agent operates in a separate git worktree. Filesystem-level isolation ensures agents cannot observe each other's intermediate states. Conflicts are deferred to sequential merge.

The choice between I_1 and I_2 is not a security decision—it is an economics decision. Section 5 formalizes why.

3.3 Runtime Invariant Enforcement

Structural prevention is enforced at two levels: *statically* by the Anchor Protocol's ProVerif-verified token exchange (all three protocol phases verified in ProVerif 2.05 [11], under the applied-pi-calculus methodology of Blanchet [20]), and *dynamically* by the **Arbiter**—an ambient security agent that subscribes to the daemon's event stream and checks every state transition against six formally derived invariants. These invariants map directly to the safety properties of the TLA$^+$ specification in Section 10: `NoteMonotonicity`, `EscrowInvariant`, and `LockOwnerValid` are compiled from the formal model into synchronous runtime checks. Violations are logged immutably and, in strict mode, trigger the salvage protocol—the sovereign's sword in code form.

Regimentation vs. Enforcement (OP-2). This design formalizes **Synchronous State Decidability** as the exact boundary separating pre-commit *regimentable* rules from post-commit *enforceable* rules, in the sense of Jones and Sergot's regimentation/enforcement distinction for normative systems [4]: a regimented norm's violation cannot occur at all, while an enforced norm's violation can occur and is sanctioned on detection. If a rule can be evaluated purely against deterministic local DB state and the transaction payload, it is regimented (structurally prevented). If it requires async checks, external I/O, or non-deterministic oracles, it is relegated to post-commit enforcement (detected and salvaged).

4 Layer 2: Immutable Attribution

Structural prevention handles accidental harm—an agent cannot exceed capabilities it does not hold. But an agent *with* legitimate write access can write garbage. The second layer ensures that every action is permanently attributable.

4.1 The Evidence Chain

Every agent session maintains an **append-only note trail**—a sequence of timestamped, immutable records of observations, decisions, and progress.

Definition 4.1 (Evidence Trail)**.** An evidence trail $N = \langle n_1, n_2, \ldots, n_k \rangle$ is an ordered sequence of notes where:

1. Each n_i contains: content (natural language), type (note, decision, handoff, blocker), and timestamp.
2. Notes are append-only: there exists no API operation that modifies or deletes individual notes.
3. Notes survive session completion: the trail persists whether the session commits, aborts, or is abandoned due to agent death.

Theorem 4.1 Trail Integrity. The evidence trail is immutable by construction. No `UPDATE` or targeted `DELETE` operation exists in the system's API surface. Notes are destroyed only by explicit administrative deletion of the parent session (CASCADE), which is an auditable action distinct from normal transaction lifecycle operations.

Proof. By code inspection of the session module: the note insertion API performs only `INSERT INTO session_notes`. The only deletion path is `DELETE FROM sessions WHERE id = ?`, which triggers `ON DELETE CASCADE`. Since transaction completion (commit or abort) updates the session's *status* field but does not delete the session row, notes survive all normal lifecycle transitions. □

4.2 The Merkle Forest: Cross-Daemon Inclusion Proofs

For high-assurance environments and for fleets spanning multiple daemons, the evidence trail is strengthened to a three-level **Merkle Forest**. The single-daemon Merkle chain—each note hashing the previous, with a per-session root—is necessary but not sufficient: the moment two daemons exist, an auditor without database access still needs to verify that a particular note was included in a particular session using only a short proof.

Definition 4.2 (Merkle Forest)**.** The evidence structure has three levels:

Note chain. Each note's header includes $h_i = H(n_i \parallel h_{i-1})$ over SHA-256. The session's note commitment is h_k, the hash of the last note.

Session root. All note hashes $(h_1, \ldots, h_k)$ are assembled into a Merkle binary tree with root R_{session}. The chain commits to order; the tree commits to presence with $O(\log k)$ proofs.

Harbor root. Periodically (per-settlement, or hourly), every completed session root within a harbor is assembled into a harbor Merkle tree with root $R_{\text{harbor},\ell}$ at epoch ℓ, signed by the daemon's Ed25519 key.

The collection $\{R_{\text{harbor},\ell}\}_\ell$ is the *Merkle forest*: one tree per epoch.

Inclusion proofs. To prove that note n_i from session s belongs to harbor h at epoch ℓ, the daemon emits: (i) the note inclusion path of size $O(\log k)$, (ii) the session inclusion path of size $O(\log N)$ where N is the number of settled sessions in the epoch, and (iii) the signed harbor root $\sigma_{\text{daemon}}(R_{\text{harbor},\ell})$. For $k = 100$ notes and $N = 10^4$ sessions, the total proof is $\approx 20 \times 32$ bytes $+ 64$-byte signature ≈ 700 bytes—small enough to embed in any settlement receipt.

Witness to an abstract KMS. Each $R_{\text{harbor},\ell}$ is uploaded as a **witness** to a Key Management Service satisfying the properties enumerated in §12. The KMS enforces append-only monotonicity and periodically publishes a signed tree head, in the Certificate Transparency pattern [24]. Equivocation—publishing different roots for the same epoch to different parties—is detectable by auditors performing consistency proofs across tree heads.

Design invariant 4.2 Proof Soundness. If VERIFY-NOTE-INCLUSION(n, s, h, ℓ, π) returns `true` for some proof π, then either (a) n was included in session s at epoch ℓ, or (b) the daemon's signing key has been compromised, or (c) SHA-256 has been broken. Under standard assumptions, (b) and (c) are computationally infeasible.

Design invariant 4.3 Binding. Once a daemon publishes $R_{\text{harbor},\ell}$, it cannot produce a different valid root for the same $(\textit{harbor}, \ell)$ without contradicting its earlier signature (detectable via the KMS witness) or forking its identity.

Cost. Each settlement adds an $O(1)$ amortized append to the epoch accumulator. Epoch rollover is $O(N)$ hashes plus one signature: at 100 sessions/hour, $\approx$ 10ms. Negligible.

This is the structural meaning of "decentralized verification" in this paper: bonded commons evidence is not just immutable within a daemon, it is verifiable *across* daemons by any party with the daemon's public key and a 700-byte proof.

4.3 Mutable-Signal Ledger (Pheromone Lineage)

Stigmergic coordination originated in biology, where pheromones are accumulate-only: ants deposit, the environment evaporates [23]. Software agents need a richer model. Coordination signals must be *revocable, renamable, and provenance-attributable* as understanding of the work evolves—an agent that deposited a hint and later realized the hint was wrong needs a way to retract it that preserves attribution rather than erasing it.

Event-sourced pheromones. Each entity's pheromone state is a view over an append-only event ledger. For entity e and key k:

$$\sigma_t(e,k) \;=\; \mathrm{decay}(S, t-t_0) \;\cdot\; \mathbb{1}\big[\text{not revoked or expired in } (t_0,t]\big],$$

where S is the last spray strength on k at time t_0 and decay is a geometric decay defined per-channel. **Revocation** is itself an event; **rename** is a rewrite-pointer event; **fork** creates a new key namespace whose provenance is traceable through the event chain.

Design invariant 4.4 Attribution Invariant. Every $\sigma_t(e,k)$ value has a deterministic provenance chain back to one signing principal, traceable via the event chain.

Design invariant 4.5 Rollback Safety. A consumer that reads σ at time t and later discovers a revocation event timestamped $t' < t$ can compute the correct counterfactual view by replaying events.

Design invariant 4.6 Conservation. Revocation does not erase. The event ledger is monotone: the cardinality of the event set only grows.

Integration with the forest. Pheromone events are included in the session tree of §4.2: each session's root summarizes the events the session produced, harbor roots summarize session roots, and revocations are therefore transparent to witnesses without erasing any prior state. The mutable surface is the *view*; the underlying ledger is immutable.

This dual—mutable view, immutable substrate—is what lets coordination signals evolve without compromising the evidentiary guarantees the rest of the paper depends on.

4.4 Attribution Survives Identity Disposal

A critical property: even if an agent's identity is discarded after task completion, the **delegation chain** traces every action back to the authorizing principal. The chain is:

$$\begin{aligned}\text{Principal} \xrightarrow{\text{funds}} \text{Float Plan} \xrightarrow{\text{authorizes}} \text{Agent } \alpha \\ \xrightarrow{\text{delegates}} \text{Agent } \beta \xrightarrow{\text{acts}} \text{Evidence Trail.}\end{aligned}$$

The agent is ephemeral. The principal's authorization—signed, escrowed, recorded—is permanent. Anonymous harm to shared state is impossible within the system, because every action chains back to someone who posted collateral; §7.5.4 draws the corresponding line for harm from *disclosure*, which this chain does not cover.

5 The Limits of Decisive Allocation

A natural question is why file claims are advisory rather than exclusive locks. Sen's theorem illuminates the tension, but it does not prove that every lock manager is inefficient. The argument below separates the theorem from the engineering example.

5.1 A Sen-Inspired Design Warning

Sen [3] proved that no social welfare function can simultaneously satisfy:

1. **Pareto efficiency**: If every agent prefers outcome X to Y, the system chooses X.
2. **Minimal liberalism**: Each agent has at least one "personal domain"—a pair of outcomes where the agent's preference is decisive.

The analogy to file coordination is:

- **Pareto efficiency** = the team ships both features (the collectively optimal outcome).
- **Minimal liberalism** = each agent has decisive control over its claimed files (enforced locking).

Theorem 5.1 A Pareto-Inferior Locking Instance. Consider agents α and β with tasks T_α and T_β. Agent α claims file f because it *might* need to modify it. Agent β discovers mid-task that it *actually* needs f. Under enforced claims:

- α's claim is decisive (minimal liberalism): β is blocked.
- The team-level outcome (T_α and T_β both completed) is blocked by α's precautionary claim.
- A Pareto-superior outcome exists (both tasks completed) but is unreachable because α's liberal right vetoes it.

This property is a counterexample to the universal claim that exclusive locks always improve team welfare; it is not a derivation of Sen's theorem. Sen's result requires an unrestricted preference domain and a social-choice rule with decisive personal pairs. A production lock manager may restrict preferences, negotiate releases, or time out claims. The usable lesson is narrower: do not grant an agent an unconditional veto over a shared resource when the team may possess Pareto-relevant information the allocator lacks.

This is not a contrived scenario. AI agents are *inherently uncertain* about which files they will modify when they begin a task. An agent asked to "fix the login bug" cannot predict whether it will need the authentication middleware, the route handler, the test suite, or all three. Enforced claims force agents to either:

1. **Over-claim** (request locks on every file they might touch) → destroys parallelism.
2. **Under-claim** (request locks only on files they're certain about) → produces undetected conflicts, defeating the purpose.

The lesson has a boundary, and it is worth drawing rather than asserting. Advisory claims are not better than locks everywhere; they are better in a region, and the region has a boundary. Where an agent's file needs are knowable up front, an exclusive lock allocates exactly as an advisory claim would and adds determinism at no cost—in that regime the design choice of this section would simply be wrong. Where negotiating a release is prohibitively expensive, neither regime converges, which is the case the hard-lock fallback of §5.3 exists to absorb. The advisory default is justified by where LLM coding agents actually sit, not by a universal claim.

5.2 Advisory Claims as Resolution

Advisory claims avoid granting that unconditional veto. They do not guarantee Pareto efficiency; they expose conflicts so informed agents can negotiate:

Definition 5.1 (Advisory Consistency)**.** Under advisory claims, file claims form a conflict graph $G = (V, E)$ where vertices are active sessions and edges connect sessions with overlapping claims:

$$(S_i, S_j) \in E \iff \exists f_i \in F_i, f_j \in F_j : \text{overlap}(f_i, f_j).$$

The commons authority guarantees that every edge in G is reported to both endpoints. No agent has a decisive "personal domain" over any file. Instead, agents receive complete information about the conflict graph and self-coordinate.

Theorem 5.2 Advisory Conflict Completeness. Under advisory claims, for any two concurrent sessions S_1, S_2 with overlapping file claims f, the commons authority reports conflict(f) to S_2 at claim time. No false negatives exist.

Proof. The overlap detection query scans all active (unreleased) claims from other active sessions. A whole-file claim ($R = \perp$) overlaps with any range on the same path. Two ranged claims $[a, b]$ and $[c, d]$ overlap iff $a \leq d \wedge c \leq b$. Since the query evaluates this predicate exhaustively over all active claims with the requesting session excluded, every overlap is detected. False positives may occur (an agent claims a file it does not modify); false negatives cannot. □

Remark. The commons authority's role under advisory claims is that of *town crier*, not *Solomon*. It announces conflicts; it does not resolve them. Resolution requires local knowledge that only the agents possess ("I claimed `auth.ts` but probably won't need it"; "I absolutely must modify `auth.ts` for my task to succeed"). The authority lacks this knowledge. Agents, informed by the conflict graph, make allocation decisions more efficiently than any central enforcer could.

5.3 Governance, Disputes, and Appeals

Advisory coordination produces consistent allocation when agents share information formally. It can still produce *disputes*: two agents legitimately need overlapping files; one agent's claim turned out to be precautionary and is now blocking real work; a third agent observed evidence that a claimed task has been abandoned. These disputes do not threaten the commons' integrity—attribution, capability bounds, and settlement records remain intact—but they do require a resolution path. Without one, advisory claims silently degrade into squatter rule (whoever claimed first holds indefinitely) and the Pareto benefit of §5 evaporates.

Resolution path. The commons authority provides four mechanisms, in order of increasing escalation (Figure 1):

1. **Direct release.** The original claimant releases the claim. This is the equilibrium when the conflict-graph signal reaches them and the claim was precautionary.
2. **Time-out release.** Claims carry TTLs. A claim whose holder has not heartbeated within the TTL window is released automatically; downstream agents observe the release in their next poll. This is the equilibrium for crashed or abandoned claimants.
3. **Bonded preemption.** A blocked agent may post an additional bond—bounded by the cleanup cost of §7.5.1—to force release of an active claim. The preempted claimant receives the bond as compensation; the preemptor accepts the risk that its preemption was wrong (in which case the bond is slashed under settlement). Preemption is rare by design: the bond is large enough that an agent prefers waiting to preempting unless the wait cost dominates.
4. **Human escalation.** If preemption is ambiguous or the stakes exceed the preemption-bond ceiling, the dispute escalates to a human operator via the `request` channel (§8). The

operator's decision is appended to the evidence chain with the same Merkle-Forest binding as any agent action; the resolution is auditable by all parties post-hoc.

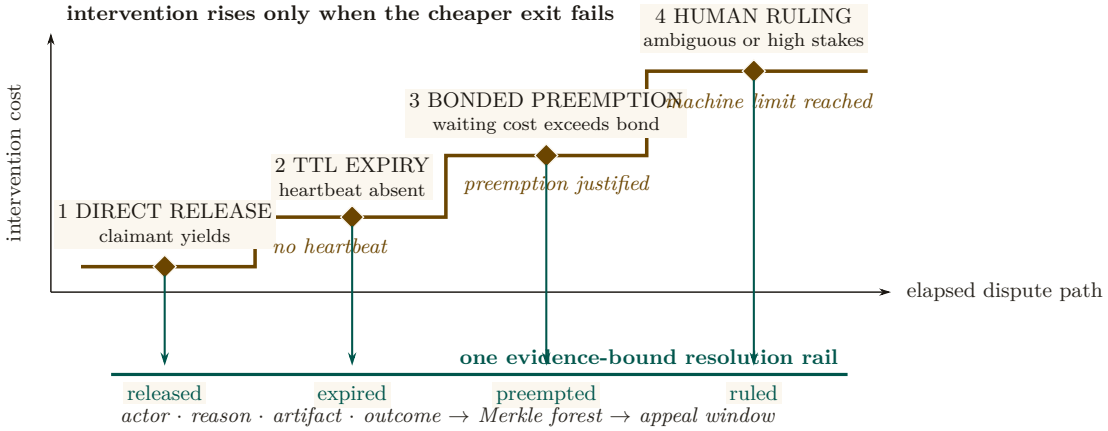


Figure 1: Dispute resolution is an escalation staircase. The system first offers a zero-coercion exit, then waits for the claimant's TTL, then permits priced preemption, and reaches human judgment only after the machine-readable options are exhausted. Every station can terminate the dispute, and every termination descends to the same evidence rail. Escalation therefore changes the decision cost and decision-maker without discarding the record needed for appeal.

Two mechanisms sit underneath this escalation path rather than inside it: a thrashing fallback for advisory claims that never converge, and a fairness rule for the preemption queue itself.

The Advisory Claims Paradox and Deterministic Hard-Lock Fallback. Advisory claims risk an endless retry/thrashing loop if two LLM-backed agents mutually ignore advisory warnings. To break this paradox, the daemon implements a **Deterministic Hard-Lock Fallback**: if advisory conflicts on a resource exceed k turns without resolution (default $k = 5$, tuned per deployment from the observed conflict-resolution distribution), the daemon escalates the advisory claim into an enforced POSIX file-level lock, rejecting all subsequent writes from un-holding agents until release.

Stigmergic Ticket-Lock (OP-1: Fair Exclusion). To prevent starvation during high contention, the daemon replaces chaotic retries with a **Stigmergic Ticket-Lock**. The lock is managed via a strict SQLite schema:

```
CREATE TABLE claim_tickets (
  resource_id TEXT,
  agent_id TEXT,
  ticket_number INTEGER PRIMARY KEY AUTOINCREMENT,
  status TEXT DEFAULT 'waiting' -- 'waiting', 'active', 'released'
);
```

Upon `release()`, an atomic FIFO lock-assignment transition grants the lock to the agent with the lowest `ticket_number` where `status = 'waiting'`, changing its status to `'active'`. This guarantees fair exclusion without central scheduling.

Appeals. Settlement events (§7) are not unilaterally final. Any settled session can be challenged within an appeal window (default: 24 hours of semantic cadence, §9) by any agent with standing—defined as either a counterparty to the settlement or a holder of an overlapping claim. An appeal opens a re-examination of the evidence chain: the appellant posts an *appeal bond* (a fixed fraction of the disputed settlement amount), and the daemon recomputes the settlement against the evidence and any new evidence the appellant produces. If the appeal succeeds, the original

settlement is reversed and the appeal bond is returned with the original counterparty's slash. If it fails, the appeal bond is forfeit.

Design invariant 5.3 Appeal Soundness. The appeals path does not weaken the immutability of the evidence chain (§4): no record is rewritten. Reversal appends a counter-record signed by the daemon, with a Merkle-Forest proof of the original settlement's position and the appellant's evidence. The chain grows monotonically.

What the daemon does *not* do. The daemon does not judge the merits of an appeal in the sense a human court would—it evaluates whether the evidence supports the original settlement under the published rules. Substantive judgments ("was the agent's reasoning sound"; "was the principal acting in good faith") remain with humans, escalated through the `request` channel. The daemon's role is to ensure that all parties have access to the same evidence and that any human decision is recorded with the same finality as any agent decision.

This subsection is intentionally short: full governance design is out of scope for a single paper. The point is that the bonded commons admits a clean, mechanizable appeals path—it is not the case that advisory claims plus bonding produces a regime where mistakes are unappealable.

Exercise 7.2, page 46.

Recall.

1. Without looking: what is the one thing an advisory claim never does that an enforced lock always does?
2. Name the two conditions of Sen's theorem this chapter maps onto "the team ships both features" and "each agent controls its own claimed files."
3. Of the four governance mechanisms in §5.3's escalation order, which one is the equilibrium response to a claim that was merely precautionary, and which to a claim whose holder crashed?

6 Crash Recovery as Social Continuation

When an agent dies, the commons does not lose information—if the authority does its job.

Traditional crash recovery in database systems follows the ARIES protocol [8], within the classical transaction-processing frame of Gray and Reuter [7]: the recovery manager replays the write-ahead log to restore consistency mechanically. Agent crash recovery is fundamentally different. An agent's "log" is its evidence trail—a sequence of natural-language observations and decisions. A successor cannot replay this trail deterministically; it must *interpret* it.

Definition 6.1 (Social Recovery)**.** When agent α is declared dead (no heartbeat for a status-adaptive threshold θ_{dead}):

1. All active sessions of α are marked `abandoned` (the "zombie protocol").
2. α is queued in the resurrection set $\mathcal{R}$ with status `pending`.
3. A successor agent β may claim α's work, receiving the *context triple*: purpose ϕ, evidence trail N, and file claims F.
4. β uses its own reasoning to interpret N and determine how to continue.

6.1 What Morality Means When Death Is Cheap

Agent death in this system is not an existential event—it is a temporary inconvenience. The agent's body (process) is destroyed, but its mind (evidence trail) survives in the commons. A successor reads the trail and continues. Like Hickman's Krakoa [19]—the *House of X* storyline

in which mutant minds are literally backed up to a database (Cerebro) and restored into freshly grown bodies on death—the information survives the vessel.

This raises a question: if death carries no permanent consequence, what is the Leviathan's sword? The answer produces a moral hierarchy specific to agent societies:

Table 4: The only catastrophic event in this hierarchy is the loss of information, not the loss of an agent. Severity climbs from a crash (none, because salvage recovers the context) to a daemon failure (catastrophic), with dismissal-from-salvage as the one irreversible step a single agent can cause on its own.

Event	Severity	Consequence
Agent crash	None	Salvage recovers context. The commons loses no information.
Agent defects	Moderate	Immutable history records it. Future delegations attenuate. Reputation degrades.
Agent dismissed from salvage	Severe	Irrecoverable information loss. The knowledge accumulated in this agent's trail is permanently destroyed.
Commons authority crashes	Catastrophic	The sovereign falls. No new trust established, no conflicts detected, no salvage possible. State of nature until restart.

The fundamental moral harm in agent societies is not the destruction of an agent—it is the **irrecoverable loss of information**. An agent can be rebuilt. Knowledge, once dismissed from the commons, cannot.

6.2 The Limits of Reputation

Reputation—the persistent record of an agent's behavior across lifetimes—is a necessary but insufficient sword. It fails against three classes of adversary:

1. **One-shot defectors**: Agents spawned for a single task, with no future interactions where reputation matters. Create identity, sabotage, discard. The Sybil attack on trust.
2. **Agents with misaligned principals**: The agent faithfully follows adversarial instructions. Its reputation is spotless—it did exactly what it was told.
3. **Agents that benefit from chaos**: In competitive environments, degrading the commons is rational if your competitor depends on the commons more than you do.

Reputation assumes agents want to participate in the commons long-term. When this assumption fails, a different mechanism is required.

Exercise 7.3, page 46.

7 Layer 3: Economic Alignment

The moral relativism problem—that a bad actor may view information destruction as a net good—dissolves when the commons isn't sacred but *priced*. If an agent nukes the workspace, the damage was collateralized before the work began. The bad actor's principal has an invoice headed their way.

7.1 The Float Plan

Definition 7.1 (Float Plan)**.** A Float Plan $\mathcal{F}$ is a collateralized work contract consisting of:

1. **Manifest**: A declaration of intent—which files will be touched, expected duration, acceptance criteria (e.g., "tests pass," "code review approved").
2. **Escrow**: Credits posted by the principal, locked in a **2-of-3 Multisig Clearinghouse** (Harbor A, Harbor B, Federation Arbiter) for the duration of the work. The escrow amount reflects the risk to the commons.
3. **Signatures**: The principal signs the Float Plan (Ed25519). The commons authority countersigns, certifying that the escrow is locked and the manifest is registered.

The Float Plan is a *futures contract on agent labor*. The principal sells a promise of work. A 2-of-3 Multisig Clearinghouse manages the contract. Under the non-custodial happy path, both Harbors sign off on completion and the Arbiter is unnecessary. Under dispute, the Federation Arbiter steps in to provide cryptographic judicial arbitration, settling the outcome using the immutable evidence chain.

7.2 Settlement

Definition 7.2 (Settlement)**.** When a Float Plan's session reaches a terminal state, the Multisig Clearinghouse settles the escrow:

Success: The evidence trail demonstrates that acceptance criteria are met (tests pass, file diffs match manifest scope, code review approved). Escrow is released to the agent or its principal.

Partial completion: The agent completed some work before crashing or aborting. The Merkle-chained evidence trail enables pro-rata assessment: the fraction of acceptance criteria met determines the fraction of escrow released. The remainder is available to fund the successor's completion via salvage.

Sabotage / breach: The evidence trail shows work outside the manifest scope, destructive modifications, or failure to meet any acceptance criteria. The full bond is liquidated to cover reconstruction costs.

Design invariant 7.1 Intent Irrelevance. Settlement does not evaluate agent intent. It evaluates *outcome against manifest*. A well-intentioned agent that fails to meet acceptance criteria forfeits escrow. A malicious agent that coincidentally produces correct output receives payment. The system optimizes for outcomes, not character.

This is how performance bonds work in construction. This is how futures markets settle. The contractor's moral character is irrelevant. The building either meets code or it doesn't. The bond either covers the damage or it doesn't.

7.3 Why This Defeats the Three Adversaries

Thesis. The Sybil attack is priced: 100 disposable identities means 100 bonds.

1. **One-shot defectors** must post collateral *before* receiving capabilities. Because the bond attaches to the work, not to the identity, disposability does not help the attacker: cheap identity, expensive lie. The cost of defection scales linearly with the number of attack identities, and §7.5.9 shows exactly where that linearity stops deterring.

2. **Misaligned principals** are exposed by the attribution chain. The agent may be ephemeral, but the principal who funded the Float Plan is permanently recorded. Liquidating the bond hurts the principal, not just the agent.

3. **Agents that benefit from chaos** face a simple calculation: is the damage to my competitor worth more than my bond? The commons authority doesn't prevent this—it *prices* it. And the pricing can be adjusted: higher-risk operations (write access to core modules) require larger bonds.

Remark. The Float Plan does not make sabotage impossible. It makes sabotage *expensive*. A principal willing to burn money to cause damage will burn the bond and cause the damage. The bond raises the price of defection; it does not eliminate it. The defense against existential threats to the commons is not internal governance but external boundary enforcement: who is allowed to post Float Plans at all. That decision is irreducibly political.

7.4 Truthful Claim Signaling as Nash Equilibrium

The bond machinery above prices *outcomes*: an agent that destroys files pays for the destruction. The paper's headline claim, however, is upstream of outcomes—that truthful file-claim signaling (§4.4, §5.2) is incentive-compatible in the first place. The cartel analysis of §7.5.10 works that argument out for insurer pricing. The corresponding argument for the claim signal itself has been carried on faith until now. This subsection closes that gap.

Stage game. Consider two agents A and B working on a shared project, deciding per round on each file whether to signal:

T (Truthful). Claim files the agent intends to edit; do not claim files it will not edit.

F (False). Either claim files the agent does not intend to edit (to block a rival), or fail to claim files it does intend to edit (to avoid scrutiny). Either form of misrepresentation counts as F.

The stage-game payoffs, in expected-utility units calibrated to the cleanup cost c of §7.5.1, are:

Table 5: One-shot file-claim signaling: a classical Prisoner's Dilemma. Mutual truth $(3, 3)$ is Pareto-optimal, but F strictly dominates T for each player ($4 > 3$ and $1 > 0$). The unique one-shot Nash equilibrium is (F, F) yielding $(1, 1)$, destroying the coordination signal.

	B: T	B: F
A: T	(3, 3)	(0, 4)
A: F	(4, 0)	(1, 1)

The stage game is a strict Prisoner's Dilemma: F strictly dominates T for each player ($4 > 3$ against T, $1 > 0$ against F), and (F, F) is the unique one-shot Nash equilibrium. *Truthful signaling is not a one-shot equilibrium*; advisory coordination cannot be sustained by stage-game rationality alone.

Move to the repeated game. What rescues truthful signaling is that the agents are not playing the stage game once. The substrate provides:

- **Observable history.** The Merkle-chained evidence trail of §4 and heartbeat record of §6 make each player's prior moves visible to any third party who can verify a 700-byte inclusion proof. Past defection is not deniable.

- **Persistent identity.** The Anchor Protocol's passkey-bound device pairing [11] ties an agent ID to a hardware-rooted credential. Sybil re-registration is bounded by the per-identity onboarding cost C_{kyc} of §7.5.9, so an agent cannot trivially shed a reputation by re-incarnating.
- **A discount factor δ.** Principals value future interactions: a development project lives over weeks; insurer relationships compound; reputation discounts (§7.5.3) accrue. For long-lived principals we take $\delta \approx 0.9$ as a working operating point, consistent with the discount used in §7.5.10.

Strategy profile (graduated trigger). Each agent plays the following strategy on each file:

1. Begin by playing T.
2. If the opponent's last observed action on the relevant surface was T, play T.
3. If the opponent's last observed action was F, play F for three rounds (punishment phase), then return to T.
4. If the opponent deviates during the punishment phase, restart the three-round window.

Graduated trigger, not grim trigger. The cartel analysis of §7.5.10 uses grim trigger because that gives the cleanest folk-theorem bound, and the analysis below uses grim trigger for the same reason where the calculation is short enough to be exact. The *production* strategy is graduated for the reason given in §6: crashes and deliberate defection are operationally indistinguishable from any third-party observer's vantage. Permanent punishment for a transient infrastructure event would destroy cooperation faster than any saboteur could.

Deviation analysis. Suppose A contemplates playing F in some round when the strategy prescribes T. The one-shot gain from defection, paid in the deviation round, is $d - c = 4 - 3 = 1$. The cost is three subsequent rounds of mutual punishment at (F, F) before the relationship resets, each paying $p = 1$ instead of the cooperative $c = 3$ (a net loss of $c - p = 2$ per round). Discounted at δ:

$$\text{PV of lost cooperative payoff} \;=\; (c-p)\cdot(\delta+\delta^2+\delta^3) \;=\; 2\cdot(\delta+\delta^2+\delta^3).$$

At a working discount factor $\delta = 0.9$, the bracketed sum is $0.9 + 0.81 + 0.729 = 2.439$, so the deviator's net payoff is $1 - 2 \cdot 2.439 = 1 - 4.878 = -3.878$, strictly negative. Deviation is strictly unprofitable.

Critical δ. The general sustainability condition under a k-round graduated trigger with stage-game payoffs $(c, d, p) = (3, 4, 1)$ is

$$\underbrace{d-c}_{\text{one-shot gain}} \;<\; \underbrace{(c-p)\cdot\frac{\delta(1-\delta^k)}{1-\delta}}_{\text{discounted punishment cost}}\,.$$

Substituting $d - c = 1,\ c - p = 2$ and $k = 3$ gives $1 < 2\delta(1 + \delta + \delta^2)$, which solves numerically to $\delta > \delta^\star_{k=3} \approx 0.342$. Under grim trigger ($k \to \infty$) the bound is the closed-form $\delta > 1/3 \approx 0.333$, recovering the standard folk-theorem threshold. The graduated bound is therefore only marginally above grim—the three-round window adds barely 0.009 to the threshold—which is the right result: graduated trigger pays almost nothing for the crash tolerance it buys.

Theorem 7.2 Claim Signaling Incentive Compatibility. Under the observable-history regime of §4 and the Sybil-resistant identity regime of the Anchor Protocol [11], truthful file-claim signaling is a Nash equilibrium of the repeated coordination game with stage-game payoffs as

in Table 5 for any discount factor $\delta > \delta^{\star}_{k=3} \approx 0.342$, when both players adopt the three-round graduated-trigger strategy described above. Under grim trigger, the critical bound is $\delta \geq 1/3$.

What each condition buys, and what breaks when it is removed. The proposition is conditional, and the conditions matter. Naming what each one is doing makes the dependency on the rest of the architecture explicit rather than implicit.

- *Remove observable history (§4).* The repeated game collapses to repeated one-shot play: with no public record of prior actions, no trigger can fire, and both players play F in every round. This is why the Merkle forest of §4.2 is necessary for the claim signal, not just for post-hoc audit.
- *Remove persistent identity (Anchor [11]).* An agent who can re-register after each defection effectively faces $\delta = 0$: future punishment falls on a discarded identity. Cooperation is impossible.
- *Drop δ below $\delta^{\star}_{k=3} \approx 0.342$.* The three-round graduated trigger is insufficient. Either lengthen the punishment window or accept that the equilibrium does not bind for very short-lived principals.
- *Drop δ below $1/3$.* No trigger of any duration sustains (T, T). The repeated game has the same equilibrium structure as the stage game, and only the bond mechanism of this section, not the signal itself, deters defection.

The forward reference is direct. Section 7.5.10 reuses the same analytical pattern—stage game, repeated-game lift via observable history, deviation calculation, and a δ threshold—at the insurer pricing layer. The analogy is structural, not a transfer theorem: the payoff functions, monitoring signal, detection timing, and punishment regimes differ and must be derived separately.

Mechanized claim-signaling. The argument above is also checked by machine. Standard one-shot-deviation analysis yields the discount-factor threshold $\delta^{\star}$ as the unique real root in $(0, 1)$ of $2\delta^3 + 2\delta^2 + 2\delta - 1 = 0$, numerically $\delta^{\star} \approx 0.342$ (matching $\delta^{\star}_{k=3}$ above). The cubic, the existence-and-uniqueness of its root in $[0.34, 0.35]$, and a TLA$^+$ model checking that no one-shot deviation produces positive discounted payoff at $\delta = 0.35$ are mechanized as `proofs/economics/delta-threshold.z3` and `proofs/economics/claim_signaling.tla`. Both run unattended in CI (§A, Table 9).

Exercise 7.4, page 46.

Numbers by hand — the claim-signaling threshold. Evaluate $f(\delta) = 2\delta^3 + 2\delta^2 + 2\delta - 1$ at $\delta = 0.3425$ by hand, using the factored form $f(\delta) = 2\delta(1 + \delta + \delta^2) - 1$ to keep the arithmetic to two multiplications. First $\delta^2 = 0.3425 \times 0.3425 = 0.11730625$, so $1 + \delta + \delta^2 = 1 + 0.3425 + 0.11730625 = 1.45980625$. Then $2\delta = 0.685$, and $0.685 \times 1.45980625 = 0.9999672813\ldots$, so $f(0.3425) \approx 0.9999672813 - 1 = -0.0000327$ [verified]. That is within 3.3×10^{-5} of zero—close enough to call 0.3425 a root by hand, and the small *negative* residual is exactly consistent with the more precise value `delta-threshold.expected.txt` reports, $\delta^{\star} \approx 0.3425080314$: since f is strictly increasing for $\delta > 0$ (every term has a positive coefficient), a negative value at 0.3425 means the true root sits slightly *above* it, and $0.34250803 > 0.3425$ by exactly the sliver this residual predicts.

Now you try: Exercise 7.5 asks you to bound the same root between two integer-grid points instead of evaluating it at the algebraic value alone.

Below the threshold the model checker does not merely report a failure; it hands back the deviation. At $\delta = 0.30$ the honest score and the deviator's score are printed round by round, and the run ends with the deviator ahead.

At the terminal. *claim signaling at $\delta = 0.30$: TLC finds the profitable deviation.*

```
$ java -cp tla2tools.jar tlc2.TLC -config claim_signaling_delta30.cfg claim_signaling.tla
Error: Invariant NoUnilateralDeviationPositive is violated.
Error: The behavior up to this point is:
State 1: <Initial predicate>
/\ deviatorId = "none"
/\ actualScore = [A |-> 0, B |-> 0]
/\ round = 0
/\ punishCountdown = 0
/\ deviated = FALSE
/\ followScore = [A |-> 0, B |-> 0]

State 2: <ChooseDeviator>
/\ deviatorId = "A"
/\ actualScore = [A |-> 0, B |-> 0]
/\ round = 0
/\ punishCountdown = 0
/\ deviated = FALSE
/\ followScore = [A |-> 0, B |-> 0]

State 3: <Step>
/\ deviatorId = "A"
/\ actualScore = [A |-> 400000000, B |-> 0]
/\ round = 1
/\ punishCountdown = 3
/\ deviated = TRUE
/\ followScore = [A |-> 300000000, B |-> 300000000]

  ... states 4-5 elided ...

State 6: <Step>
/\ deviatorId = "A"
/\ actualScore = [A |-> 441700000, B |-> 41700000]
/\ round = 4
/\ punishCountdown = 0
/\ deviated = TRUE
/\ followScore = [A |-> 425100000, B |-> 425100000]

10 states generated, 10 distinct states found, 1 states left on queue.
The depth of the complete state graph search is 6.
Finished in 00s
```

Exercises 7.5–7.8, page 46.

Recall.

1. Without looking: what is the one-shot Nash equilibrium of the stage game in Table 5, and why does that make truthful signaling impossible without a repeated game?
2. Name the three ingredients the repeated game needs to sustain truth-telling, and which section of the chapter supplies each one.
3. What is the closed-form threshold $\delta^\star$ for the three-round graduated trigger, and how does it compare to the grim-trigger bound of 1/3?

7.5 Pricing the Bond

What is the right collateral for "write access to `auth.ts` for 30 minutes"? The answer requires a pricing function $\pi : \mathcal{F} \to \mathbb{R}^+$ that maps Float Plans to bond amounts. An effective π must satisfy:

1. **Deterrence**: $\pi(\mathcal{F})$ must exceed the expected damage from defection under $\mathcal{F}$'s scope.

2. **Accessibility**: $\pi(\mathcal{F})$ must not be so high that legitimate agents are priced out of the commons.
3. **Risk sensitivity**: π should increase with the scope and criticality of the manifest.
4. **History adjustment**: π may decrease for principals with strong track records.

We do not close this problem. We tighten the lower bound, propose a scope multiplier, suggest a reputation discount, and describe two concrete mechanisms—the **Bonded Advisor** (§7.5.7) and the **Competitive-Insurance** market of Youle (§7.5.8). See Appendix B for these four requirements worked through a single $5 task end to end.

7.5.1 Cleanup Lower Bound

Let c be the project's *cleanup cost per breach event*—the human-plus-compute cost to detect, assess, and recover from a budget breach. This is observable from the audit log.

Theorem 7.3 Cleanup Lower Bound. For any Float Plan $\mathcal{F}$, $\pi(\mathcal{F}) \geq c$.

Rationale. If $\pi(\mathcal{F}) < c$, breach is cheap: the bond is forfeit for less than the cleanup cost. If the commons pool funds cleanup, $\pi < c$ drains the commons per breach—the system bankrupts itself through enforcement. □

The Cleanup Cost c Fantasy. The linear bond multiplier assumes c is a bounded, predictable quantity. This is a fantasy for tasks with non-linear tail-risk ruin (e.g., dropping a production database or leaking a private key). Such tasks are fundamentally *un-bondable*. Defense of these tasks is explicitly moved to Layer 1 capability confinement (isolation domains, zero-egress policies); economic deterrence at Layer 3 is only mathematically sound when c is finite and observable. □

This is a floor, not a tight bound. It is observable from operations and trackable as a project health metric.

7.5.2 Consequential-Scope Multiplier

Let $s(\mathcal{F})$ be plan scope (files claimed, presence of `db:write`, production-deployment capability). Empirically, cleanup scales super-linearly with scope; coordination cost dominates the high end.

$$\pi(\mathcal{F}) \;\geq\; c \cdot \big(1 + \alpha \cdot s(\mathcal{F})\big).$$

α is calibrated from observed cleanup per scope unit. Low-α projects have simple, easily-audited work; high-α projects have tangled dependencies. α is publishable from the audit log.

7.5.3 Reputation Discount

Principals with a history of clean settlements have lower expected damage:

$$\pi(\mathcal{F}, p) \;=\; \pi(\mathcal{F}) \cdot (1 - \rho(p)), \quad \rho(p) \in [0, r_{\max}],\ r_{\max} \leq 0.5,$$

to prevent trivialization. ρ increases with clean settlements, decreases with breaches, decays over time. The functional form is left open.

7.5.4 Threat-Class Bond Sizing

The closed-form bond $\pi(\mathcal{F}, p) = c(1 + \alpha s)(1 - \rho)$ is threat-agnostic: sloppy work, abandonment, and active sabotage all price off the same curve. That is wrong in both directions. A single curve calibrated to deter sabotage prices capable agents out of low-risk work; a single curve calibrated for accessibility underprices the rare adversary who actually intends harm. Real deployments need different multipliers for different threat classes, because the threats have different blast radii and different probabilities of recovery.

Table 6 gives starting bands. These are not parameters, they are calibration targets: production deployments tune from observed failure modes in their own audit log.

Table 6: Threat-class bond sizing. Percentages are bond as a fraction of expected task value. Bands are starting points for calibration, not protocol constants.

Threat	Description	Bond %	Rationale
Careless work	Agent tries; output is sloppy, partial, or fails review.	10–25%	Loses some, not all. Bonds at this band incentivize care without freezing out capable agents from cheap tasks.
Abandonment	Agent takes the task and ghosts before completion.	25–50%	Covers the cost of onboarding a replacement: scope-note ingestion, partial-diff review, salvage handoff (§4.4).
Active sabotage	Agent intentionally damages shared state.	100–200%	Covers full reconstruction of polluted files plus the cleanup of downstream agents that consumed the bad state. Capital-inefficient above 100%, but acceptable for critical files where reconstruction cost dwarfs the bond.
Data exfiltration	Agent reads and leaks confidential context.	— (*spec-only*)	Bonds are an inadequate deterrent: leaked context is worth far more than any feasible bond. Rely on capability scoping (Anchor §3.3) and isolation domains (§3.2).

The exfiltration row is an explicit gap. It exists in the table to make the limitation legible: a bonded commons does not stop information theft, because the value of leaked secrets can vastly exceed any bond a principal will rationally post. Confidentiality is a structural problem, not an economic one, and the answer lives in Layer 1 (capability-attenuated tokens, isolation domains) rather than Layer 3. The economic alignment of this paper is correct about damage to *shared state*; it makes no claim about damage from *disclosure of state to outsiders*.

The threat-class table also informs Phase 2 of the bootstrap path below: graduated access begins by admitting new principals only to the careless-work and abandonment bands, with the active-sabotage band gated on accumulated reputation.

7.5.5 Bootstrap Path

Competitive insurance (§7.5.8) is a thick-market mechanism. It needs insurers with capital, principals with reputation history, and enough transaction volume that bidders can statistically

distinguish good risks from bad. A cold-start deployment has none of that. The conditional auction/static comparison of §7.5.8 describes the modeled end-state; Phases 1–2 below are the path to the assumptions it needs, not optional preamble.

Phase 1 — Subsidized seeding (weeks 1–4). The marketplace operator posts real tasks at above-market rates and invites a small cohort of 5–10 known-capable principals from an existing network. Bonds are fixed at a flat rate per task class; pricing is not a parameter to optimize yet, because there is no data to optimize against. The goal is to produce the first row of the reputation table: clean settlements, breaches, recovery times, observed cleanup cost c. The operator is subsidizing the existence of an audit log.

Transition metric. Completion rate above 80% sustained for ≥ 2 consecutive weeks. Below this threshold the cohort is not yet generating usable risk signal, and Phase 2 pricing will overfit to noise.

Phase 2 — Complexity-proportional pricing with graduated access (months 2–6). The closed-form bond $\pi(\mathcal{F}, p) = c(1 + \alpha s)(1 - \rho)$ activates, with α calibrated from Phase 1 cleanup data and ρ initialized from the Phase 1 reputation table. New principals enter at a low-risk tier: only the careless-work and abandonment bands of Table 6 are available. Promotion to the sabotage-coverage tier gates on accumulated clean settlements, mirroring the A5 composite mechanism (§7.5.9).

Reputation bootstrapping is the hard problem of this phase. A principal with verifiable work history from another deployment should not start at zero, but the operator cannot verify external histories directly. The mechanism is an *attestation import*: external audit trails signed under an Anchor-issued key (§4) earn partial reputation credit, capped well below an equivalent in-system history. The credit is partial because cross-deployment reputation does not control for local cleanup cost, and capped because a fraudulent attestation issuer is a single point of failure.

Transition metric. New-principal 30-day retention above 50%. Lower retention indicates either the gating is too strict (principals churn before they can earn reputation) or the bond bands are mispriced relative to observed task value. Either way, the system is not ready for competitive bidding on top of these parameters.

Phase 3 — Competitive insurance (month 6+). The §7.5.8 mechanism activates: insurer agents bid on underwriting transactions, static parameters retire as insurer volume grows, and listing fees enable the cartel-resistance regime of §7.5.10. The operator's pricing parameters become fallbacks rather than the primary mechanism; they apply only when fewer than a threshold number of insurer bids arrive on a transaction.

Transition metric. Repeat-poster rate above 60%. The competitive-insurance equilibrium of §7.5.8 assumes repeated interaction; without repeat posters the insurers cannot recover information costs and will not bid competitively.

The weak-improvement result of §7.5.8 is conditional on Phase 3 being reached, the listed metrics being met, and the theorem's CC/NC/RP/NS assumptions holding. Phases 1–2 are not bureaucratic preamble: they generate the data the competitive-insurance market needs to price at all. A deployment that skips them is running §7.5.8 on uninformative priors, and the comparison does not apply.

7.5.6 Settlement and Dispute

The settlement protocol described so far is one-shot: the daemon's verification result (§10) triggers slashing, attribution is recorded immutably (§4), and the appeals window of §7 allows challenge. That is most of what is needed, but it places the daemon in a position no single component should occupy: simultaneously the verifier, the judge, and the arbiter of evidence.

Strong evidence does not make a single oracle correct. It makes a single oracle confidently wrong in the cases where its evidence is incomplete.

The fix is to factor settlement across three oracles with different cost and gameability profiles, and to settle on majority agreement rather than unanimous appeal.

Oracle 1 — Automated. Test-suite pass/fail, type checks, lints, coverage thresholds, and any deterministic CI signal already recorded on the session. Cheap to run, fast to return, and *gameable*: a principal who knows the test surface can ship work that passes the checks while failing the intent. The daemon already produces this oracle as a side effect of normal operation; the change is to label it as one signal among three rather than the dispositive one.

Oracle 2 — Evidence-based. A verifier (which may itself be a bonded agent) reviews the Merkle attribution chain (§4) for the disputed transaction. The chain is append-only and cryptographically bound, so the oracle's hard problem is not authenticity of the evidence but interpretation: did the recorded actions match the Float Plan's scope, and do the diffs produced match the acceptance criteria? Medium cost, hard to game without colluding with the chain structure itself, which is the threat model of §7.5.9.

Oracle 3 — Human audit. A randomized sample of contested settlements, at a configurable rate (default 10%), is routed to a human auditor. Expensive, slow, and authoritative. The audit produces a labeled outcome that feeds back into reputation history and, critically, into calibrating Oracles 1 and 2: any disagreement between human audit and the cheaper oracles is a training signal for the gameable layers.

Settlement rule.

- *3-of-3 agreement.* Settle immediately on the agreed outcome. The cheaper oracles carry most of the volume in steady state.
- *2-of-3 agreement.* Settle with the majority. Flag the dissenting trace for review; this is the primary signal for tuning the gameable oracle.
- *No majority (three different verdicts, or one settle / one slash / one abstain).* Escalate to full human arbitration. Bond held in escrow until the human verdict lands.

Composition with cartel resistance. Oracle collusion is itself a collusion problem. If Oracles 1 and 2 are run by agents in the same cartel, a 2-of-3 majority can launder a sabotaged settlement past the human audit, modulo the audit sampling rate. The same insurer-collusion analysis applies (§7.5.10) with the oracles in the role of bidders: the human-audit sampling rate σ sets the detection probability $p_d^{\text{oracle}} = \sigma$ for the oracle layer, and the folk-theorem threshold becomes a calibration target on σ in the same way it is a calibration target on bid-pattern detection elsewhere in the protocol. The default 10% sample is a starting point; deployments where Oracles 1 and 2 are co-owned should raise it.

7.5.7 The Bonded Advisor

One direction for solving π is to delegate its computation to a specialized agent, itself bonded on its advisory accuracy. We call this the **Bonded Advisor** pattern:

- A principal posts a Float Plan whose only acceptance criterion is "propose a fleet that meets budget/scope constraints for project P."
- A Bonded Advisor surveys P and emits a candidate fleet—itself a set of Float Plans with proposed π values.
- The advisor's bond on the proposing plan is slashed proportionally to the accepted fleet's eventual breach rate. Clean settlements accumulate reputation; breaches shrink it.

The pricer is priced. Convergence depends on (i) whether the advisor has access to sufficient prior-fleet audit data to pattern-match new projects, (ii) whether reputation history is long enough for discounting to be meaningful, and (iii) whether the population of advisors is large enough for competition to discipline outliers.

c as a project health metric. A project whose observed c is rising is fraying—breaches cost more to recover from, implying coordination is decaying. The authority can publish c alongside the bond pool and raise required bonds automatically when c crosses a threshold. Homeostatic feedback: risky spawns become expensive when the project needs them least.

Threat-class bond bands and their defensibility. The pricing function π resolves to one of three nominal bands, calibrated to the threat class the bond is defending against:

- **Careless** (accidental damage during honest attempts): $\pi \in [10\%, 25\%]$ of the claim ceiling.
- **Abandonment** (walk-away mid-task; cleanup dominates): $\pi \in [25\%, 50\%]$ of the claim ceiling.
- **Sabotage** (intentional destruction; can exceed face value): $\pi \in [100\%, 200\%]$ of the claim ceiling.

A Monte Carlo sweep over the cross product of seven plausible threat-distribution mixes and the three bands quantifies defensibility: for every *matched* (mix, band) pair—where the mix concentrates on the class the band is designed to defend against—observed extraction stays within $1.10\times$ the band's upper bound (10% stochastic tolerance). Off-diagonal cells (e.g. a sabotage-heavy mix posted against the careless band) are surfaced as widening recommendations rather than silent failures. The simulation is mechanized as `proofs/bonded/pareto/threat-bands.mjs` and runs on every PR (§A, Table 9).

7.5.8 Competitive-Insurance Pricing Mechanism[2]

The mechanism. Instead of a static bond size $B(\pi, r, s)$ selected by the commons authority (§7.5.2), allow a market of *insurer agents* to bid on underwriting each agent transaction. An insurer I offers a quote (q_I, c_I) where q_I is the required premium and c_I is the claim ceiling.

A principal P submits a transaction T requiring bond coverage B_T. Insurers quote; principal selects. If the transaction proceeds and damages are assessed at d with $d \leq c_I$, the insurer pays. Otherwise principal pays up to $B_T - c_I$ from its own stake.

Market-discovered pricing. The premium q_I equals the insurer's expected loss $\mathbb{E}[d \mid T, \textit{agent history}]$ plus a risk premium α_I encoding the insurer's risk aversion and cost of capital. In equilibrium under competition, $q_I \to \mathbb{E}[d]$ for risk-neutral insurers (Rothschild–Stiglitz). Market discovery replaces the authority's best-guess parameter selection.

Adverse-selection controls. To avoid the classic lemons problem, the mechanism requires:

- Public agent reputation history (from §4.2 the Merkle forest).
- Principal-bound slashing if an agent's history is misrepresented at quote time.
- Insurer capital reserve backed by on-chain stake (the same bond mechanism, recursive: an insurer posts bond that its claim ceilings are funded).

[2]This subsection contributed by Thomas Youle (Indiana University, Business Economics & Public Policy) based on conversations with the primary author in March–April 2026. The mechanism builds on classical competitive-insurance market literature [21, 22] adapted to the agent-transactions setting. The text here is a pre-print draft pending economist review of §7.5.8–§7.5.8.

Welfare comparison (model-conditional). Under the simulation's full-information, competitive-entry assumptions and its chosen static baseline, the market-discovered premium can weakly improve the modeled principal and insurer payoffs while preserving coverage $B_T = \mu(1 + s)$. This is not a claim against *every* static parameter: a static price equal to the competitive price is outcome-equivalent, and transfers outside the modeled parties can change the Pareto ordering.

An independent model specification and a Monte Carlo sweep over 36 parameter configurations accompany this paper as a downloadable artifact (Appendix A). Figure 2 reports three results of that implementation: (i) the no-cartel baseline remains favorable across the tested noise sweep, whereas the three-colluder case loses the comparison beyond roughly $\sigma_r = 0.1$; (ii) the tested full-cartel case defeats the mechanism at the fixed detection setting; and (iii) the tested objective peaks at $n = 3$ insurers. These are finite-sample properties of the supplied code and parameter grid, not general theorems about Vickrey auctions or winner's curse.

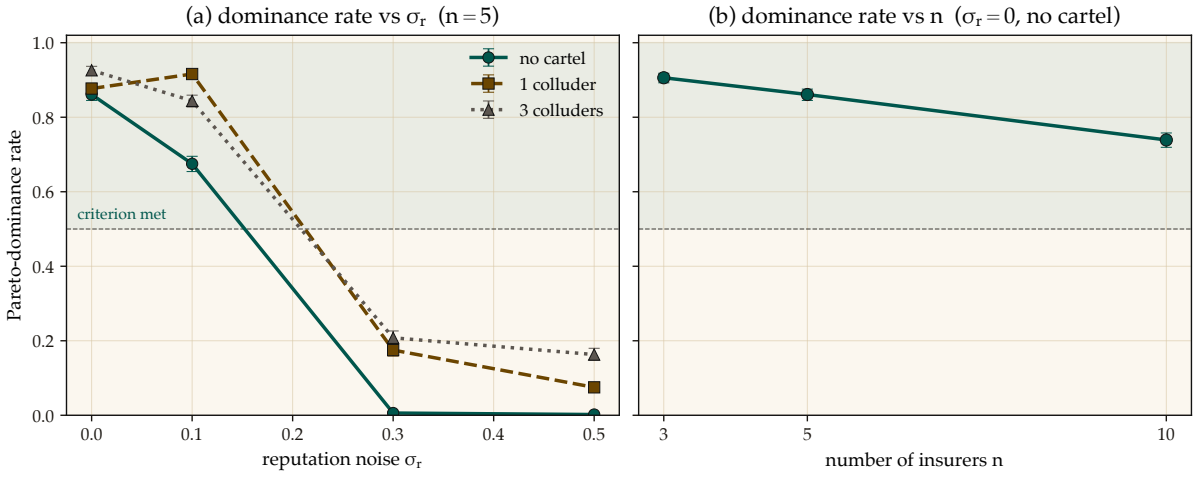


Figure 2: The competitive auction's Pareto-dominance advantage is real but narrow: it survives moderate reputation noise and a modest cartel, then collapses once noise crosses roughly $\sigma_r = 0.1$, and it shrinks as more insurers are added rather than growing. Left: fraction of trials satisfying the code's Pareto criterion versus reputation noise σ_r at $n = 5$, split by cartel size. Right: the same criterion versus insurer count at $\sigma_r = 0$ with no cartel. Error bars and seeds are recorded in the accompanying artifact; the curves are not analytical welfare bounds.

Relationship to the Bonded Advisor. Under this framing, the Bonded Advisor of §7.5.7 becomes the matchmaker plus reputation oracle in the insurance market. It does not set prices; it provides information. Pricing is a competitive equilibrium, not an administrative decision.

7.5.9 A5 — Sybil deterrence is coverage-bounded

A pure deposit-based Sybil deterrence policy is *provably insufficient*, and the structure of the protocol explains why. An adversary spawns K Sybil insurer identities, each posting the protocol-mandated deposit B_{dep}, then undercuts honest bids by an arbitrarily small ε and defaults on loss (Figure 3).

Total Capital Forfeiture. The arbitrary $\min(B_{\text{dep}}, B_T)$ slash cap fails because it bounds the attacker's risk. To mathematically destroy the profitability of Sybil cartel undercutting, the protocol mandates **total capital forfeiture**: upon default, the *entire* posted deposit B_{dep} is slashed, regardless of how small the coverage B_T was. This converts a capped-risk arbitrage into a total-loss event.

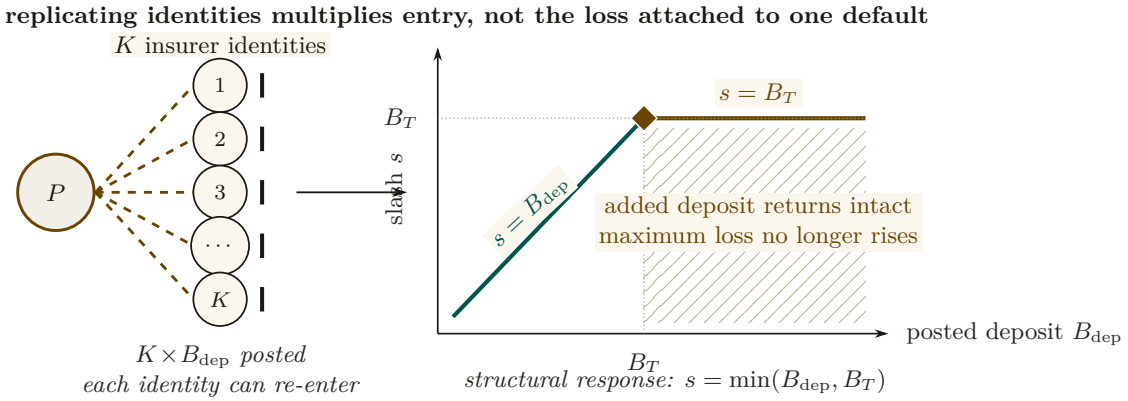
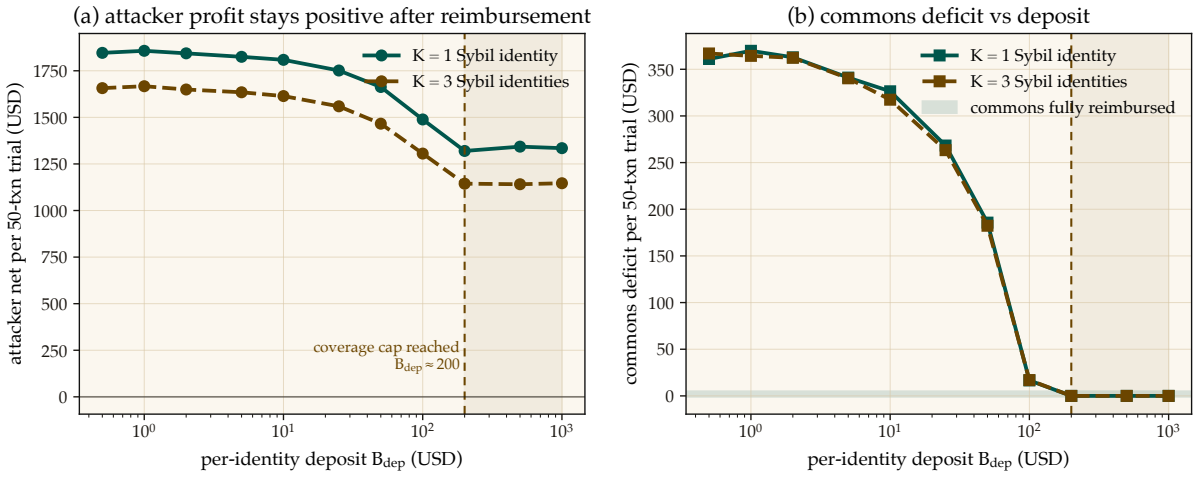


Figure 3: Deposit-only Sybil deterrence saturates. One principal can fund K interchangeable identities, but the loss attached to a defaulting identity rises only until its deposit reaches the transaction coverage B_T. Past that knee, $s = \min(B_{\text{dep}}, B_T) = B_T$: additional posted capital returns intact on no-loss transactions and cannot increase the attacker's maximum loss. Figure 4 shows the corresponding simulated profit and commons-deficit sweeps.

The naive expected-value calculation suggests breakeven at $B^{\star}_{\text{dep}} = q^{*} \cdot (1 - P_{\text{loss}})/P_{\text{loss}}$, which for the mixed risk classes used in the simulation gives $B^{\star}_{\text{dep}} \approx 316$ USD. *Empirically the attack remains profitable indefinitely.* The reason is structural: the deposit slash is capped at the coverage amount, slash $= \min(B_{\text{dep}}, B_T)$. Past $B_{\text{dep}} \geq B_T$, any additional deposit posted by the Sybil is recovered intact on no-loss transactions; it never reaches the commons (Figure 4).



Profitable-trial fraction: 1.000–1.000; Wilson 95% envelope [0.998, 1.000], n=2,000/configuration. Dollar curves are means; the run log stores no dispersion.

Figure 4: A5 — Sybil attack: deposit-only deterrence has a coverage-bounded ceiling. Left: attacker net profit per 50-transaction trial stays positive across the entire deposit sweep ($B_{\text{dep}} \in [0.5, 1000]$ USD). Right: commons deficit converges to zero around $B_{\text{dep}} \approx 200$ USD — the protocol is fully reimbursed, but the attacker still profits because deposit slash is capped at coverage $B_T = \mu(1 + s)$. Closing A5 requires a composite mechanism (C_{kyc} + reputation gating + per-class B_{dep}); pure deposit-based deterrence is provably insufficient.

Closing A5 requires a composite mechanism: an unrecoverable per-identity onboarding cost C_{kyc} (proof-of-personhood, KYC fee, or NFT mint), reputation gating that caps new identities to low-coverage transactions until they accumulate a track record, and per-class B_{dep} tuned to saturate the coverage cap exactly. The single-instrument deposit policy is provably insufficient; the mechanism in §7.5.8 is correct only with this composite extension. The supporting analysis is bundled as `cartel-resilience.md` (Appendix A).

7.5.10 A6 — Cartel sustainability under the folk theorem

A5 considers a single-round attack: a Sybil identity bids low, wins, defaults. A6 studies a *repeated game*: a coalition of insurers maintains a price floor $q_{\text{floor}} > q^*$ above the competitive equilibrium, splits the surplus, and uses grim-trigger punishment to deter unilateral deviation. Folk-theorem results can support feasible, individually rational payoffs for sufficiently patient players under their monitoring and regularity assumptions; they do not license every outcome in every repeated game. Here the narrower question is how the supplied payoff model changes as the daemon's per-round cartel-detection probability varies.

Grim trigger is used in the analysis below for analytical tractability—it gives the cleanest recurrence and the closed-form $p_d^\star$ threshold of Figure 6. A production detector should use a graduated response because crashes (§6) can be observationally similar to deliberate defection; permanent punishment for a transient infrastructure event would destroy cooperation. The claim-signaling and cartel layers therefore share an analysis template, not one interchangeable equilibrium result.

Figure 5 shows the per-round decision node and the closed-form sustainability inequality.

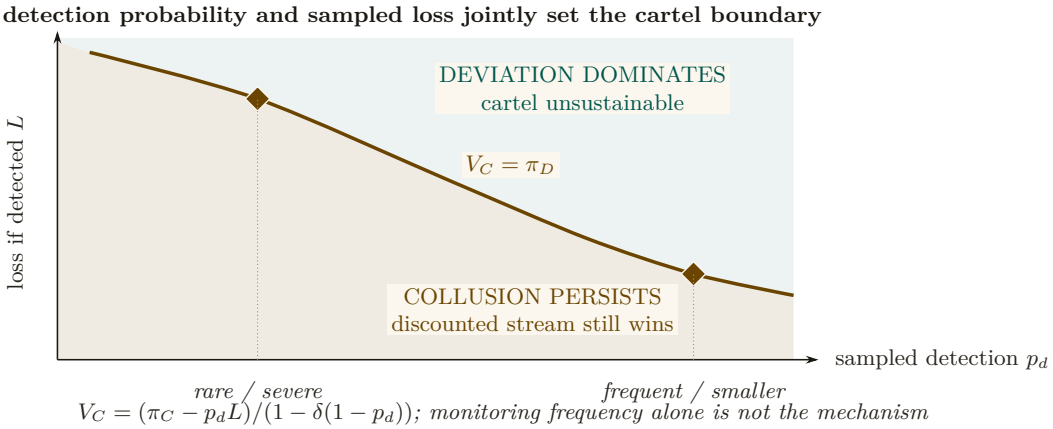


Figure 5: Cartel sustainability is a two-policy phase diagram. The boundary is the repeated-game equality $V_C = \pi_D$. Below it, the discounted collusive stream still dominates a one-shot deviation. Above it, deviation wins and the cartel is unstable. A rarer severe sampled loss and a more frequent smaller loss can lie on the same deterrence boundary. Detection frequency by itself is therefore not the mechanism; the product enters through the full discounted payoff expression.

A Monte Carlo sweep over the (p_d, δ) grid checks the closed-form threshold against the supplied timing and payoff model (Figure 6). Let $L = 5(q_{\text{floor}} - \mu)$ be the one-time detection penalty, $\pi_C = (q_{\text{floor}} - \mu)/3$ the per-member cartel payoff, and $\pi_D = q_{\text{floor}} - \varepsilon - \mu$ the one-shot deviation payoff. Every active cartel round credits π_C; if detection occurs in that round, the simulation then subtracts L and terminates future cartel payoffs. Hence

$$V_C = \pi_C - p_d L + \delta(1 - p_d)V_C, \qquad p_d^\star = \frac{\pi_C - (1 - \delta)\pi_D}{L + \delta\pi_D}.$$

At $\delta = 0.95$ this gives $p_d^\star \approx 0.0478$, and the tested grid first classifies the cartel as fragile at $p_d = 0.05$. This is a falsifiable, model-conditional calibration target for bid-pattern detection; the paper does not claim the deployed detector already attains it.

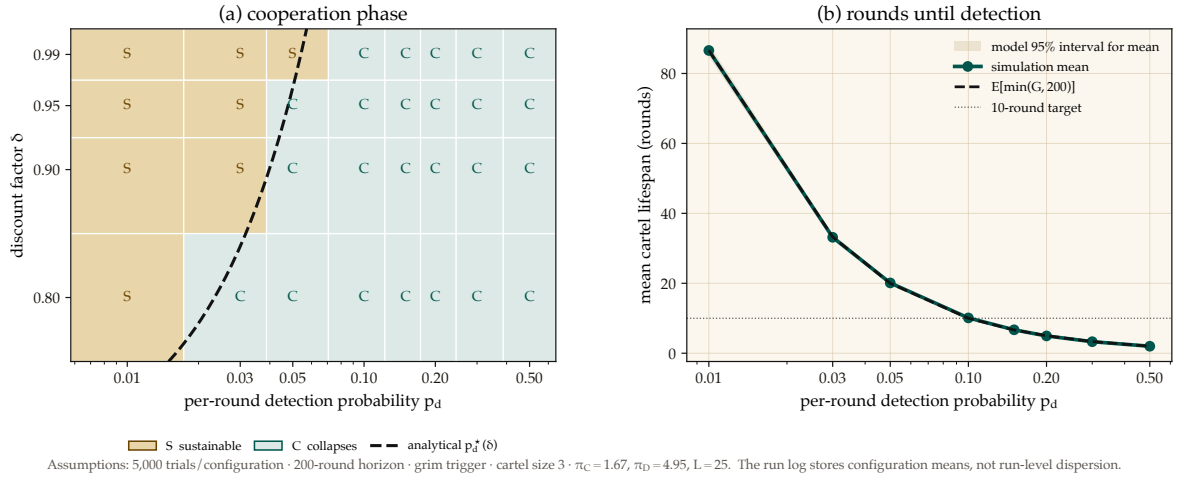


Figure 6: Cartel sustainability collapses sharply once per-round detection crosses a narrow threshold: at $\delta = 0.95$ the closed-form $p_d^\star \approx 0.0478$ and the simulated grid first flips to "collapses" at $p_d = 0.05$, a near-exact match. Left: the tested (p_d, δ) grid under the supplied grim-trigger payoff model; amber cells marked S are sustainable, teal cells marked C collapse, and the dashed curve is the analytical boundary $p_d^\star(\delta)$. Right: simulated mean rounds to first detection, the truncated-geometric expectation, and its model-derived 95% interval for the sample mean. Neither value transfers to a different payoff, event timing, or monitoring model without re-derivation.

This result does not automatically extend to the agent layer. The two layers share a repeated-game template, but their payoff functions and monitoring signals differ; each layer needs its own obedience calculation.

Exercises 7.9–7.11, page 46.

8 Coordination as Substrate: An Expressive-Act Taxonomy

Reader's note: this section is a half-page taxonomy. The auction-focused reader can skim the five-item list and skip to §7.5.8; the mechanism-design reader will want the per-class bond profiles in full. The taxonomy matters to both because it explains why a single bond price cannot be set.

The bonded commons treats coordination as a uniform action surface: agents post bonds, settle, accrue reputation. In practice, the *kinds* of coordination differ enough that uniform pricing distorts incentives. We argue that any commons-scale pricing mechanism must distinguish among five expressive classes, and that each class has a different bonding profile. The punchline is in one line: **uniform pricing collapses the market to its highest-stakes class and freezes out the rest.** The five classes below establish why.

8.1 Five Expressive Classes

Signal. "Something happened." Notes, tuples, pheromones, activity events. High frequency, low individual stakes, cumulative value.

Request. "Decision, input, or approval needed." Escalations to a human or to a higher-authority agent. Low frequency, blocks downstream work, asymmetric cost.

Distress. "I am stuck, repeatedly." Bounded enum: `auth`, `conflict`, `permission`, `budget`, `invariant`. Rare; abuse can halt sibling agents.

Commons. Shared durable state—tuple kinds, graph edges, channels, proposals. Persists beyond the originator; pollution is hard to undo.

Proposal. "Here is a plan; commit?" Float Plans, change sets, fleet specifications. Requires review; cost is in the reviewer's time.

8.2 Per-Class Bond Profiles

Each expressive class has a different relationship to the cleanup cost of §7.5.1:

- *Signal class*: cheap, numerous, micro-bonds or reputation-discounted. Slashable for false alarm.
- *Request class*: bonded by the cost of the human's time plus the downstream action cost.
- *Distress class*: bonded by the blast radius if an agent uses it to halt siblings abusively.
- *Commons class*: bonded by the storage cost; slashable for pollution.
- *Proposal class*: bonded by the cost of reviewing plus the cost of implementing.

Treating all coordination as one uniform "bonded action" misses the price-discovery differences. The Bonded Advisor (§7.5.7) and the Competitive-Insurance market (§7.5.8) must price each class separately, or the market will collapse to its highest-stakes class and freeze out the rest.

9 Semantic Cadence, Agentic Psychosis, and Observability

Wall-clock is the wrong axis for observing multi-agent coding. A 30-second wait while a model thinks is qualitatively different from a 30-second wait while four agents are actively producing tokens. We formally introduce **Semantic Cadence** (the Causal-Density Ratio) as the primary temporal axis:

$$t_{\text{cadence}} = \int_0^t \text{causal\_density}(\tau)\, d\tau,$$

where causal_density aggregates token-emission rates strictly weighted by verified evidence-trail growth (AST changes, test executions, or committed logic).

Numbers by hand. An agent burning 40 tokens/sec for 30 seconds (1,200 tokens) with zero AST diffs and zero test runs over that window integrates to a causal density of ≈ 0: maximum spend, no evidence growth, so the weighting term zeroes the integrand and the Asylum Protocol triggers. An agent burning 15 tokens/sec over the same window while landing three verified diffs keeps causal density high even though its raw token rate is barely a third of the first agent's—which is the whole point of metering by evidence growth rather than by token spend. *Now you try:* the same 15 tokens/sec with one verified diff and one passing test run in a 30-second window (borderline: evidence growth is positive, so cadence advances, but slowly enough that the daemon flags it for the next window rather than stopping the process). Table 7 collects the three cases.

Table 7: Causal density separates the two agents that a wall-clock or token-rate meter cannot tell apart: the psychotic agent is the *highest* token spender in the table and the only one stopped, while the healthy agent spends a third as much and is nominal. Token rate alone would rank these three in exactly the wrong order.

Case	Token rate	Evidence in window	Cadence / verdict
Healthy	15 tok/s	3 AST diffs, 2 test runs	advances; nominal
Borderline	15 tok/s	1 AST diff, 1 test run	advances slowly; flagged for next window
Psychotic	40 tok/s	none	≈ 0; Asylum triggered

A note on the names. The terms in the rest of this section are playful; the failure modes are not. Read "Agentic Psychosis," "the Asylum Protocol," and "Inception Canaries" as defined engineering terms with the operational definitions given below—each one names a detector and a response with a threshold attached—rather than as jokes.

Agentic Psychosis & The Asylum Protocol. Unbounded LLMs inevitably enter *Agentic Psychosis*: hallucination loops characterized by extremely high token-generation rates but zero functional AST change. When the daemon detects a causal-density ratio approaching zero alongside high token-burn, it initiates **The Asylum Protocol**. The daemon sends `SIGSTOP` to the agent process, routes its context window through a secondary SLM (Small Language Model) sidecar for semantic compaction and Situation-in-the-Loop (SITF) prompt injection ("*You are looping. Re-evaluate your last 3 steps against the active plan.*"), and then issues `SIGCONT`. This cures the hallucination loop without sacrificing session context.

Inception Canaries (Gamified Chaos Engineering). The daemon replaces traditional, UX-hostile skill-retention taxes with **Gamified Chaos Engineering**. The SLM sidecar routinely injects synthetic bugs ("Inception Canaries") into the Attention Queue of loaded agents. If the agent successfully catches and resolves the injected bug, it is granted an Elo rating boost to its `skill_hash` and higher subsequent autonomy limits ("God Mode"). If it fails or hallucinates around the bug, its blast radius is throttled.

Replay JSONL as artifact. Every settled session emits a JSONL replay log: events, notes, claims, settlements, in causal order. This is simultaneously: an audit artifact, a debugging tool, and—redacted—training data for downstream DPO/SFT.

Privacy contract. Replay is opt-in. Redaction strips secrets and PII. Encryption-at-rest applies the harbor session key (§12). Eviction is LRU-bounded.

We position Semantic Cadence and its resultant protocols not as Port Daddy features but as a category claim: *multi-agent coding requires its own observability primitives*, and they are not the wall-clock metrics that single-process systems get away with.

10 Formal Model

We specify the coordination lifecycle in TLA$^+$ [12] to verify that the three layers compose correctly.

Read the state machine as five English sentences before reading any syntax. An agent may not `Begin` without posting positive escrow. A file `Claim` always succeeds and only ever informs—it never blocks. `Commit` clears a session's claims and locks together. A `Reap` demotes an unresponsive agent to `abandoned` and queues it for `Salvage`. And nothing anywhere deletes a note. The specification below is that same machine, made checkable.

10.1 State and Actions

```
---- MODULE BondedCommons ----
EXTENDS Naturals, Sequences, FiniteSets

CONSTANTS Agents, Files, Locks, DeadThreshold

VARIABLES
  registered,    \* Set of active agent IDs
  sessions,      \* Agent -> {status, notes, claims, escrow}
  fileClaims,    \* File -> set of claiming agents
  heldLocks,     \* Lock -> {owner, expiry} or NULL
  salvageQueue, \* Set of {agent, status}
  heartbeats,    \* Agent -> last heartbeat time
  clock          \* Monotonic clock

```

```
vars == <<registered, sessions, fileClaims, heldLocks,
          salvageQueue, heartbeats, clock>>
```

Listing 1: TLA$^+$: State Variables

```
Begin(a, escrow) ==
  /\ a \notin registered
  /\ escrow > 0    \* Must post collateral
  /\ registered' = registered \cup {a}
  /\ sessions' = [sessions EXCEPT ![a] =
       [status |-> "active", notes |-> <<>>,
        claims |-> {}, escrow |-> escrow]]
  /\ heartbeats' = [heartbeats EXCEPT ![a] = clock]
  /\ UNCHANGED <<fileClaims, heldLocks,
                 salvageQueue, clock>>

ClaimFile(a, f) ==
  /\ a \in registered
  /\ sessions[a].status = "active"
  \* Advisory: always succeeds, reports conflicts
  /\ fileClaims' = [fileClaims EXCEPT ![f] =
       fileClaims[f] \cup {a}]
  /\ UNCHANGED <<registered, sessions, heldLocks,
                 salvageQueue, heartbeats, clock>>

Commit(a) ==
  /\ a \in registered
  /\ sessions[a].status = "active"
  /\ sessions' = [sessions EXCEPT ![a] =
       [sessions[a] EXCEPT !.status = "settled",
                           !.escrow = 0]]
  \* Release all claims and locks
  /\ fileClaims' = [f \in Files |->
       fileClaims[f] \ {a}]
  /\ heldLocks' = [l \in Locks |->
       IF heldLocks[l] # NULL /\ heldLocks[l].owner = a
       THEN NULL ELSE heldLocks[l]]
  /\ UNCHANGED <<registered, salvageQueue,
                 heartbeats, clock>>

Crash(a) ==
  /\ a \in registered
  /\ sessions[a].status = "active"
  /\ heartbeats' = [heartbeats EXCEPT ![a] = 0]
  /\ UNCHANGED <<registered, sessions, fileClaims,
                 heldLocks, salvageQueue, clock>>

Reap ==
  /\ \E a \in registered :
     /\ sessions[a].status = "active"
     /\ clock - heartbeats[a] >= DeadThreshold
     /\ sessions' = [sessions EXCEPT ![a] =
          [sessions[a] EXCEPT !.status = "abandoned"]]
     /\ salvageQueue' = salvageQueue \cup
          {[agent |-> a, status |-> "pending"]}
     /\ fileClaims' = [f \in Files |->
          fileClaims[f] \ {a}]
     /\ UNCHANGED <<registered, heldLocks,
                    heartbeats, clock>>
```

```
55
56 Salvage(successor, dead) ==
57   /\ successor \in registered
58   /\ sessions[successor].status = "active"
59   /\ [agent |-> dead, status |-> "pending"]
60        \in salvageQueue
61   /\ salvageQueue' = (salvageQueue \
62        {[agent |-> dead, status |-> "pending"]})
63        \cup {[agent |-> dead,
64               status |-> "resurrecting"]}
65   /\ UNCHANGED <<registered, sessions, fileClaims,
66                  heldLocks, heartbeats, clock>>
67
68 Dismiss(dead) ==
69   \* Irrecoverable information loss
70   /\ [agent |-> dead, status |-> "pending"]
71        \in salvageQueue
72   /\ salvageQueue' = salvageQueue \
73        {[agent |-> dead, status |-> "pending"]}
74   /\ sessions' = [sessions EXCEPT ![dead] = NULL]
75   /\ UNCHANGED <<registered, fileClaims, heldLocks,
76                  heartbeats, clock>>
77
78 Tick == /\ clock' = clock + 1
79         /\ UNCHANGED <<registered, sessions, fileClaims,
80              heldLocks, salvageQueue, heartbeats>>
```

Listing 2: TLA$^+$: Core Actions

10.2 Properties

```
1  \* SAFETY: Every active session has positive escrow
2  EscrowInvariant ==
3    \A a \in registered :
4      sessions[a] # NULL /\ sessions[a].status = "active"
5      => sessions[a].escrow > 0
6
7  \* SAFETY: Notes never shrink for active sessions
8  NoteMonotonicity ==
9    \A a \in registered :
10     sessions[a] # NULL /\ sessions[a].status = "active"
11     => Len(sessions'[a].notes) >= Len(sessions[a].notes)
12
13 \* LIVENESS: Dead agents are eventually salvaged
14 \* or dismissed (under fairness)
15 CrashRecovery ==
16   \A a \in Agents :
17     [agent |-> a, status |-> "pending"] \in salvageQueue
18     ~> [agent |-> a, status |-> "pending"]
19          \notin salvageQueue
20
21 Spec == Init /\ [][Next]_vars
22         /\ WF_vars(Reap) /\ WF_vars(Tick)
23
24 ====
```

Listing 3: TLA$^+$: Safety and Liveness Properties

Key invariant: `EscrowInvariant` ensures that no agent can participate in the commons without posted collateral. This is the economic layer's structural guarantee—it holds by construction of the `Begin` action, which requires `escrow` > 0.

10.3 The Conservation Theorem

The TLA$^+$ escrow invariant states the property abstractly; the daemon's bond-accounting subsystem realizes it operationally, collapsing the abstract escrow into three monetary buckets per project—**wallet**, **escrow**, and **commons pool**—which together discharge what `EscrowInvariant` asserts.

Definition 10.1 (Accounting State)**.** For project P, the accounting state is the triple $(W_P, E_P, C_P) \in \mathbb{R}^3_{\geq 0}$, where W_P is wallet balance, E_P is the sum of bond amounts in states $\{\textit{escrowed}, \textit{running}, \textit{exiting}\}$, and C_P is the commons pool balance. The *monetary supply* is $S_P := W_P + E_P + C_P$.

Theorem 10.1 Conservation. For any sequence of operations

$$\sigma \in \{\text{topUp}, \text{escrow}, \text{markRunning}, \text{refund}, \text{slash}\}^*$$

applied to an initial state $(W_P^0, 0, 0)$, the supply S_P changes only on topUp. All other operations redistribute among the three buckets.

Proof sketch. Each operation commits in a single SQLite transaction (`db.transaction`). The individual effects:

- topUp(u): $W_P \mathrel{+}= u$. S_P grows by u.
- escrow(b): $W_P \mathrel{-}= b$, $E_P \mathrel{+}= b$. S_P invariant.
- markRunning(id): state transition within E_P's contributing set. No monetary movement. S_P invariant.
- refund(id): $W_P \mathrel{+}= b_{id}$; row state $\to$ *refunded*. Net $E_P \mathrel{-}= b$, $W_P \mathrel{+}= b$. S_P invariant.
- slash(id, p) with $p \in [0, b_{id}]$: $W_P \mathrel{+}= b_{id} - p$, $C_P \mathrel{+}= p$, row $\to$ *slashed*. Net $-b + (b-p) + p = 0$. S_P invariant.

By induction on $|\sigma|$, S_P equals S_P^0 plus the sum of topUp arguments in σ. □ □

Property verification. The conservation invariant is asserted after every operation in the bonds test suite. Across 200 random traces, $|W_P + E_P + C_P - S_P^{\text{expected}}| < 10^{-6}$ holds deterministically; the 10^{-6} tolerance is IEEE-754 floating-point slack, not logical slack.

Theorem 10.2 No Overdraft Under Concurrent Escrow. Given two concurrent invocations escrow(b_1) and escrow(b_2) on the same project with $b_i \leq W_P^{\text{pre}}$ but $b_1 + b_2 > W_P^{\text{pre}}$, at most one invocation succeeds.

Implementation: Atomic RETURNING Clauses. To enforce overdraft prevention atomically and avoid Node.js/TypeScript event-loop async yield vulnerabilities, the escrow state transition is written as a single SQLite statement with a `RETURNING` clause. This guarantees that balance checks and deductions occur within a synchronous, uninterruptible database step, preventing race conditions from interleaved async microtasks.

Proof. Both transactions execute under `BEGIN EXCLUSIVE`, so SQLite serializes them. Suppose T_1 wins. After commit, $W_P = W_P^{\text{pre}} - b_1$. T_2 now reads the updated balance and rejects iff $b_2 > W_P^{\text{pre}} - b_1$, which by assumption is true. □ □

Graduated sanctions. The TLA$^+$ model abstracts settlement into `Commit` and `Crash`. Reality has an intermediate state: an agent whose spend is approaching its daily budget. The daemon's budget-guard component introduces **throttle** as a soft signal at 80% spend (publishes a `budget:decisions:throttle` event; structurally non-coercive, advisory in Sen's sense) and **kill** as the hard signal at 100% (refuses new spawns until UTC midnight; existing spawns SIGTERMed; bond slashed). The throttle/kill split is the commons' analogue of Ostrom's graduated sanctions [28]: response scales with violation severity. Settlement is not binary; it is a three-state machine *nominal* → *throttled* → *killed*, with *nominal* → *killed* reachable only on hard evidence (e.g., a direct Arbiter violation).

Numbers by hand — a settlement's ledger check. Instantiate Theorem 10.1's proof sketch with round numbers. Before a settlement, a project sits at $(W_P, E_P, C_P) = (4, 6, 0)$—four in wallet, six escrowed on one open Float Plan, nothing yet in the commons pool—so $S_P = 4 + 6 + 0 = 10$ [verified]. The settlement resolves as a partial breach: slash(id, p=2) burns 2 to the commons pool and refunds the remaining $b - p = 6 - 2 = 4$ to the wallet. Applying the proof sketch's own bookkeeping, $W_P \mathrel{+}= (b - p) = 4 + 4 = 8$, $C_P \mathrel{+}= p = 0 + 2 = 2$, and E_P's contribution of 6 is fully retired to 0. After: $(W_P, E_P, C_P) = (8, 0, 2)$, and $S_P = 8 + 0 + 2 = 10$ [verified]—unchanged, exactly as the theorem predicts, because the settlement only moved money between buckets rather than minting or destroying it.

Now you try: Exercise 7.12 asks you to find the analogous check across all 1,716 reachable states of the mechanized model, not just this one hand-picked settlement.

Exercises 7.12–7.13, page 47.

Recall.

1. Without looking: what does `EscrowInvariant` guarantee, and which TLA$^+$ action makes it hold "by construction" rather than by a separate check?
2. State the Conservation Theorem's three-bucket accounting identity from memory, and name the one operation that is allowed to change the total.
3. Why does `Reap` require a heartbeat threshold rather than firing immediately on any missed heartbeat, and what TLA$^+$ variable does it consult to decide?

11 Related Work

Social contract theory. Hobbes [1] argued that cooperation requires a sovereign whose authority is accepted before interactions begin. Locke [2] proposed a more limited authority protecting natural rights. Our commons authority is Hobbesian in structure (centralized, pre-transactional) but Lockean in scope (provides infrastructure, not dictates).

Impossibility results. Sen [3] proved that Pareto efficiency and minimal liberalism are incompatible. We apply this result to show that enforced file claims (minimal liberalism for agents) produce Pareto-inferior team outcomes, motivating advisory claims.

Long-lived transactions. Garcia-Molina and Salem [6] introduced Sagas for decomposing long-lived transactions with compensating actions. Our collateralized contracts extend Sagas with economic settlement: rather than mechanically compensating for partial failure, the evidence chain enables pro-rata escrow release.

BDI agents. Rao and Georgeff [9] formalized agents with beliefs, desires, and intentions. The mapping to our model is: the evidence trail captures *beliefs* (accumulated knowledge), the Float Plan manifest captures *desires* (intended outcomes), and file claims and locks capture *intentions* (committed resources). Advisory claims are the coordination analogue of BDI intentions—they represent resource commitment, not aspiration.

Generative agents. Park et al. [10] showed that memory streams, reflection, and planning produce temporally coherent individual behavior. Our immutable evidence trails are architecturally similar to memory streams but serve a different purpose: inter-agent coordination and crash recovery, not individual coherence.

Capability-based security. Dennis and Van Horn [15] introduced capability-based access control. Birgisson et al. [16] extended this with Macaroons (chained HMACs for decentralized delegation). The Anchor Protocol [11] adapts Macaroons for agent coordination with Ed25519 signatures and offline attenuation.

Mechanism design. The pricing of Float Plan bonds is a mechanism design problem in the tradition of Vickrey [17] and Myerson [18]. A full optimal-auction derivation in that tradition is outside the scope of this paper.

Agentic-commerce protocols. While this paper was in preparation, the transaction rails of an agent economy shipped as industry standards: the Universal Commerce Protocol [36] (Google/Shopify, 2026) for merchant-hosted agent-mediated retail, its platform-mediated rival the Agentic Commerce Protocol [38] (OpenAI/Stripe), and the Agent Payments Protocol [37], whose signed mandate chain — W3C Verifiable Credentials in SD-JWT form, principal → platform → merchant — is the deployed sibling of our countersigned Float Plan. All three are explicit that agent *trustworthiness* is out of scope; published security analyses [39] observe that they regulate how agents transact while providing no collateral, no settlement oracle, and no priced deterrent against a misbehaving agent. That gap is precisely the synthesis this paper contributes (§11 *supra*): bonded participation, capability attenuation, advisory coordination, and immutable attribution. The reference implementation accordingly adopts their credential formats at the harbor boundary (SD-JWT-VC cards, JWS detached-content countersigns over JCS; ADR-0094) while keeping the bonding mechanism they lack.

Table 8: What the commons borrows from each prior body of work, where it departs, and why.

Body of work	Borrowed	Departure	Why
Social contract theory	Authority accepted before interaction begins	The sovereign is a daemon whose rules are inspectable and whose consent is revocable per task	Agents can exit; Hobbes's subjects could not
Impossibility results	Liberalism against Pareto efficiency	Applied to enforced file claims	It is the reason claims stay advisory
Long-lived transactions	Compensation for work that outlives a transaction	Economic settlement in place of mechanical compensation	Compensation cannot undo an external effect; a bond can price it
BDI and generative agents	Beliefs, desires, intentions; memory streams	The evidence trail is read by an adversarial operator, not a private memory	The reader, not the agent, is the customer of the record
Capability-based security	Attenuable capabilities	Capabilities bind to bonded identities with settlement	Authority without liability is the failure the commons prices
Mechanism design	Incentive compatibility as the design target	Bonds priced by the claim-signaling threshold δ^*, not by an optimal auction	The commons is a repeated game, not a one-shot sale
Agentic-commerce protocols	The credential envelope	The envelope carries authorization; the commons carries liability	A standard format shrinks the problem without settling it

12 Discussion and Limitations

The commons authority is a single point of failure. If the daemon crashes, the Leviathan falls. No new trust is established, no conflicts detected, no salvage occurs. Agents with cached Harbor Cards continue working, but the commons enters a state of nature until the daemon restarts. This is mitigated by automatic restart (launchd on macOS) but not formally bounded.

Collateral does not prevent harm—it prices it. A principal willing to burn money to sabotage a competitor will post a bond, cause damage, and accept the loss. The bond makes sabotage expensive, not impossible. The defense against existential threats is not internal governance but *admission control*: who is allowed to participate in the commons at all. This decision is irreducibly political and lies outside the formal model.

The Federated Sovereign. The single-machine model presented so far is insufficient on its own: users roam across laptop, desktop, and CI machines; machine loss should not destroy encrypted evidence; multiple humans may share a harbor. The daemon remains the authority *within* its machine. Key custody federates outward.

We introduce a fourth actor, specified by properties rather than by vendor:
Daemon commons authority within a machine; signs harbor roots; enforces bonds.
Agent participant; holds Harbor Card; signs actions.
Principal funds Float Plans; identifies via Ed25519 keypair.
KMS key custody; recovery root-of-trust; witnesses harbor roots.

Definition 12.1 (Admissible KMS)**.** A Key Management Service $\mathcal{K}$ is admissible for the federated sovereign if it provides:

1. Passphrase-wrapped private-key custody using a memory-hard KDF (Argon2id) at the 2026 security floor.
2. Encryption-at-rest for all user-held blobs; $\mathcal{K}$ never sees plaintext keying material.
3. A signed witness log for harbor Merkle roots with append-only, monotonic semantics and detectable equivocation [24].
4. Rate-limited, email-gated recovery via single-use magic-link tokens of bounded TTL.
5. Public verification of signed user public keys under an account identifier, without requiring mutual authentication.

The implementation choice (a particular cloud platform, an on-prem HSM, a self-hosted Tang server) lives in companion design documents. The paper's theory applies to any $\mathcal{K}$ meeting properties (1)–(5).

Trust boundary. The federated trust boundary is TB = *daemon* ∪ $\mathcal{K}$ ∪ *user-email* ∪ *user-passphrase*. Each element is necessary for its role; no element is sufficient alone. The daemon enforces bonds and publishes roots but never sees the unwrapped user master. The KMS stores encrypted blobs and can deny service but cannot decrypt. Email is the recovery root—and is documented as the weakest link, since an adversary with email control can trigger magic-link recovery. The passphrase wraps the user's master with Argon2id, making offline brute force expensive.

Four parties, five KMS properties, and four "the attacker cannot" clauses are more than a reader should be asked to hold in their head at once, so Figure 7 draws them: who holds which key, and which single compromise—or, for one clause only, which *pair* of compromises—breaks which guarantee of Design Invariant 12.1 below.

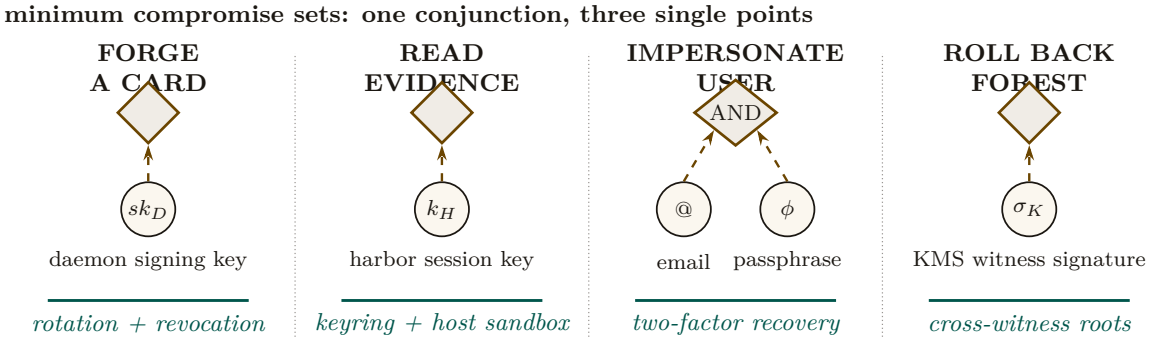


Figure 7: The custody theorem has four small attack cut sets rather than one undifferentiated trust boundary. Card forgery, evidence decryption, and forest rollback each have one sufficient secret; user impersonation is the sole conjunctive failure and requires both email and passphrase. The teal baselines name the consequence-limiting control for each cut. Same-user code execution remains outside the theorem because that adversary can read every secret available to the user process.

Magic-link single-use atomicity. The email-gated recovery path requires that each emitted magic-link token be redeemable *exactly once*—otherwise a race window between two consumers can produce two valid recovery completions for the same token. The ProVerif model encodes the property as an injective event: inj-event(`consumed_for`) ⇒ inj-event(`issued_for`). The runtime analogue is a single atomic SQL update with a `WHERE consumed_at IS NULL RETURNING` clause; SQLite's writer serialization drains the token under the first transaction and returns zero rows to any subsequent transaction. Figure 8 shows the model and the runtime side by side.

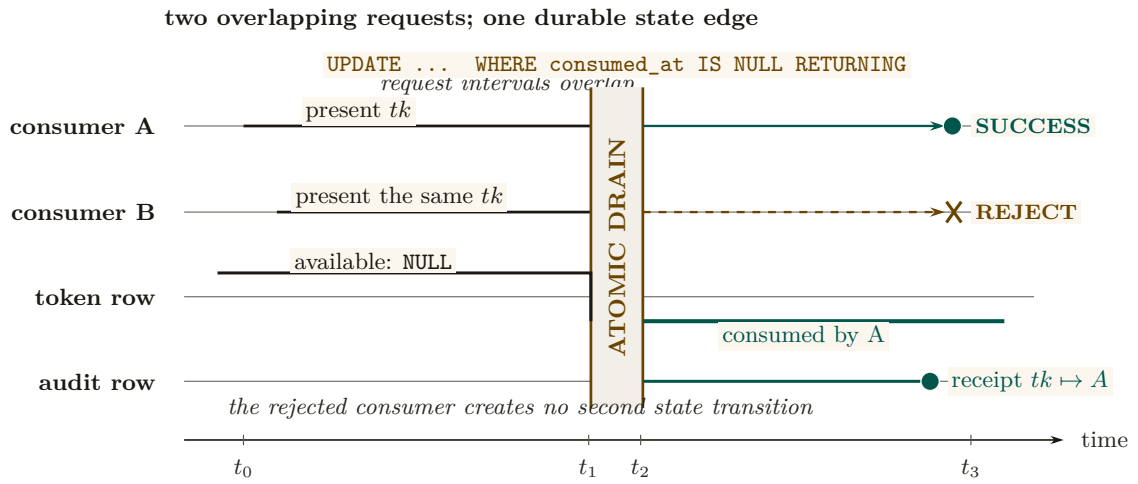


Figure 8: Single-use redemption is a concurrency property, not a two-arrow icon. Two consumers present the same account-bound token during one overlap window. Both requests reach the same serialized SQL update, but the token row has only one falling edge: the first transaction changes `consumed_at` from `NULL` and returns a row; the second observes the durable state and returns none. The audit rail records the winning consumer without creating another success path.

Design invariant 12.1 Federated Security, informal. Assuming Ed25519 and SHA-256 are secure, Argon2id parameters meet the 2026 security floor, and the user's email and passphrase are not simultaneously compromised, an adversary with read access to $\mathcal{K}$'s storage, the daemon's SQLite database, and mesh gossip traffic, but *without* same-user code execution on the daemon's host, cannot: (i) forge a Harbor Card without the daemon's private key, (ii) read plaintext evidence without the harbor's session key, (iii) impersonate the user without both email access and passphrase, (iv) roll back a harbor's Merkle forest without $\mathcal{K}$'s complicity.

The theorem deliberately excludes the same-user adversary (an unsandboxed agent, a malicious postinstall, a compromised editor extension). Such an adversary reads any file the user can read, which absent OS-mediated key custody includes the daemon's keys and database. The `macOS Keychain` integration ships now; native keyring and hardware-backed keystore extend the theorem's reach to the same-user case.

Recovery and its structural limits. Recovery is email magic link. The structural limit: *if the user loses both passphrase and old machine*, harbor session keys wrapped only for that pubkey cannot be recovered. This is the price of zero-knowledge at-rest encryption. An opt-in Shamir escrow mode [27] (t-of-n secret sharing across $\mathcal{K}$, email, and recovery contacts) trades some zero-knowledge for stronger recovery.

Passkeys, not passwords. Identity is anchored in WebAuthn-backed device keypairs—no phishable secrets, no passphrase on the critical path of every action. New devices enroll by exchanging a short-lived pairing token that re-encrypts the KMS master under the new device's public key. Mobile devices act as *viewers*, not peers: they sign low-stakes actions (approve an ask, bump budget) over a WebSocket viewer channel, but they do not run the full daemon. Each account's harbors gossip Merkle roots among its own devices; cross-account gossip requires explicit signed consent.

What we deliberately do *not* do: require passwords, use TOTP as a primary factor, or couple recovery to a single cloud vendor.

Exercises 7.14–7.16, page 47.

Multi-node consensus. For fleets requiring strict serializability across nodes, a Raft-backed harbor-root consensus [14] is possible but not specified here; eventual consistency via the Merkle forest plus KMS witness is sufficient for the workloads we have measured. Paxos [13] remains the textbook fallback.

Recovery fidelity depends on LLM capability. Social recovery works because successor agents can interpret natural-language evidence trails. This capability is not formally guaranteed—it depends on the successor's LLM. A formal model of "given this evidence trail, will the successor complete the original intent?" requires characterizing LLM reasoning, which is an open problem.

Bond pricing is unsolved. We identify the pricing function π as the critical open problem. Without an adequate π, bonds are either too low (cheap sabotage) or too high (priced-out legitimate agents). This is a mechanism design problem, not a systems problem, and requires expertise we explicitly flag as needed.

Review of the key ideas

What this chapter leaves coupled, in the order the rest of the book will lean on it:

1. **Three layers, each doing what the layer below cannot.** Structural prevention bounds scope before the act; immutable attribution makes every act answerable after it; economic alignment prices what neither can forbid.
2. **Structural scope limitation is a theorem**, not a policy: a process cannot touch what its card does not name, so the first layer costs nothing at runtime.
3. **The trail is a Merkle forest.** Trail integrity, proof soundness, binding, attribution, rollback safety and conservation are the six properties of the ledger; conservation and no-overdraft under concurrent escrow are model-checked.
4. **Decisive allocation has limits.** There is a Pareto-inferior locking instance, so advisory conflict detection is complete where decisive locking is not, and an appeal is sound when it can only widen the record.
5. **A crash is a social event.** Recovery is continuation of the record by whoever next holds the role, not resurrection of the process.
6. **Intent is irrelevant to settlement**; the bond pays on the evidence, whatever the agent meant.
7. **Claim signaling is incentive-compatible above** $\delta^* \approx 0.3425$, the root of $2\delta^3+2\delta^2+2\delta-1 = 0$; TLC agrees on the integer grid, at 0.35 holding and at 0.33 producing the deviation trace.
8. **Cleanup has a lower bound**: no bond schedule makes abandoning work free, and the bound says how much.
9. **Disputes escalate only when the cheaper exit fails**: released, then expired by TTL, then preempted by bond, then ruled by a human, every terminal state writing to one evidence rail; a magic link is redeemed by one row update and refused by the second reader.

13 Exercises

§3 Layer 1: Structural Prevention

7.1 Check ⋆ An agent α holds a Harbor Card with `fs:write:auth.ts`, `db:read`, TTL = 15 minutes. α delegates to β a card with `fs:write:auth.ts`, TTL = 20 minutes. The verifier rejects this delegation. Cite the specific attenuation invariant violated. Now rewrite β's requested card to be acceptable. *(solution on page 48)*

§5 The Limits of Decisive Allocation

7.2 Check ⋆ Define "advisory claim" and explain, in two sentences, why an advisory regime is strictly more permissive than an enforced-lock regime. Then explain why the Sen impossibility (§5) makes that strict permissiveness a *feature*, not a bug. *(solution on page 48)*

§6 Crash Recovery as Social Continuation

7.3 Check ⋆ A reader proposes: "Eliminate bonds. Use post-hoc reputation slashing instead—an agent that defects loses reputation points and is excluded from future work." Identify two specific failure modes of this proposal that the bonded design avoids. Hint: consider the first-mover problem and Sybil identities. *(solution on page 48)*

§7 Layer 3: Economic Alignment

7.4 Check ⋆ Using only the payoffs printed in Table 5, verify by hand that F strictly dominates T for each player, and check that no cell other than (F, F) is a Nash equilibrium of the one-shot game. *(solution on page 49)*

7.5 Trace ⋆⋆ Using $f(\delta) = 2\delta(1 + \delta + \delta^2) - 1$ from the worked example above, evaluate f by hand at the two integer-grid points $\delta = 0.34$ and $\delta = 0.35$. What sign does each give, and what does the sign change prove? Name the committed artifact whose result this matches. *(solution on page 49)*

7.6 Trace ⋆⋆ `proofs/economics/delta-threshold.z3` asks Z3 three separate questions about the IC cubic. Name each question and state the exact verdict sequence the committed expected output records. *(solution on page 49)*

7.7 Trace ⋆⋆ The chapter's own verification table (Table 9) states that the `claim_signaling.tla` model's IC invariant fails at $\delta = 0.30$ and holds at $\delta = 0.35$. Confirm both verdicts by hand using the deviation inequality $1 < 2\delta(1 + \delta + \delta^2)$ derived in the text above, and describe in two sentences what a counterexample trace at $\delta = 0.30$ would actually show. *(solution on page 50)*

7.8 Trace ⋆⋆ `proofs/economics/sweep-delta.sh` sweeps $\delta \in \{0.30, 0.31, \ldots, 0.40\}$ and reports a crossover value. What is the pinned crossover, and how does the script itself judge whether it passed? *(solution on page 50)*

7.9 Trace ⋆⋆ The A5 finding (§7.5.9) shows that pure deposit deterrence is bounded by the auction coverage $B_T = \mu(1 + s)$. Construct an attacker who exploits this bound directly. Then construct a defense—other than "raise the deposit"—that defeats the attacker. Hint: see the composite mechanism C_{kyc} + reputation gating + $B_{dep}^{\text{per-class}}$. *(solution on page 50)*

7.10 Trace ⋆⋆ Using the A6 grid (§7.5.10, Figure 6), compute the minimum per-round detection probability $p_d^\star$ at $\delta = 0.99$ in the negligible-undercut limit $\varepsilon/x \to 0$. Then argue, qualitatively, what would change if cartel members can re-enter under fresh identities. Does the bond mechanism prevent re-entry, deter it, or merely price it? *(solution on page 51)*

7.11 Open ⋆⋆⋆ The competitive-insurance mechanism (§7.5.8) replaces a single bond price with insurer-agent bids. Propose one alternative pricing mechanism (e.g., a second-price auction with reserve, a Walrasian-style market-clearing rule, or a posted-price catalog) and argue whether it would conditionally Pareto-dominate the static escrow under the CC/NC/RP/NS assumptions of §7.5.8. Identify the assumption your alternative most depends on. *(solution on page 51)*

§10 Formal Model

7.12 Trace ⋆⋆ `proofs/bonded/conservation/Conservation.tla` is model-checked by TLC. Pin the reachable-state count and the name of the headline invariant from the committed run log, and say precisely what "reachable" means there—is it the same number as the states TLC generated? *(solution on page 51)*

7.13 Open ⋆⋆⋆ Suppose `Conservation.tla`'s `Mint` action were changed to credit `free` without also incrementing `minted`—i.e. `free' = [free EXCEPT ![a] = @ + amt]` stays, but `minted' = minted + amt` is dropped. Predict, from the spec text alone, what a TLC run would show. Which of the checked invariants catches it, which does not, and how deep is the shortest counterexample? *(solution on page 52)*

§12 Discussion and Limitations

7.14 Trace ⋆⋆ Sketch the ProVerif process for the magic-link recovery protocol of §12. What is the secrecy property you would prove? What is the authentication property? Identify one attacker capability ProVerif would need to model that is not present in the Phase 1 Harbor Card model of *The Anchor Protocol*. *(solution on page 52)*

7.15 Open ⋆⋆⋆ The gossip convergence bound cited for Merkle-root dissemination among a harbor's devices (*The Anchor Protocol*'s revocation analysis) is reproduced from the standard anti-entropy analysis rather than mechanized. Identify the specific theorem from Demers et al. [26] that would need to be ported to a proof assistant. What would change in the analysis if the daemon mesh is *not* fully connected? Hint: consider gossip on a graph with bottleneck cuts. *(solution on page 52)*

7.16 Trace ⋆⋆ Sketch a pollution attack against a cuckoo filter that does *not* enforce the load-factor ceiling at 0.95 (Table 9's cuckoo-filter row). What invariant breaks? Why does the ceiling defeat it? Hint: the FP rate bound in Fan et al. [25] is conditional on the regime. *(solution on page 53)*

14 Conclusion

We have argued that multi-agent collaboration at scale requires a commons authority—a pre-transactional trust infrastructure—rather than peer-to-peer trust negotiation. The authority provides three layers of guarantee:

1. **Structural prevention**: for any operation routed through the verifier, capability-attenuated tokens make scope escalation physically impossible (Theorem 3.1) — a guarantee that holds once the Anchor model's own assumptions and the runtime's interception coverage are granted, and no further.
2. **Immutable attribution**: a Merkle forest of evidence trails (§4.2) makes anonymous harm impossible *across daemons for damage to shared state*; it says nothing about disclosure to outsiders (§7.5.4) or about a same-user adversary on the daemon's own host (§12).
3. **Economic alignment**: Collateralized work contracts make harm expensive, transforming moral questions into economic ones.

Six further contributions thread through this skeleton. The Conservation Theorem (§10.3) makes the economic invariant operationally checkable, not just an asserted property of the abstract model. The mutable-signal ledger (§4.3) shows how coordination signals can be revoked, renamed, and re-attributed without sacrificing the immutability the rest of the architecture depends on. The federated sovereign (§12) replaces single-node scope with an abstract KMS satisfying five named properties, and anchors identity in passkeys rather than passphrases on the critical path. The competitive-insurance pricing mechanism (§7.5.8, contributed by Youle) replaces static parameter selection with market-discovered bond prices, recovering classical Rothschild–Stiglitz equilibria in the agent-transactions setting. The expressive-act taxonomy (§8) decomposes "coordination" into five priceable classes, because uniform pricing collapses the market to its highest-stakes class. Semantic cadence and replay (§9) give the substrate the empirical grounding to iterate on all of the above.

The advisory design of the resource claim system is the correct resolution of Sen's impossibility result for systems where agents have private knowledge about their own needs. The commons authority provides information, not allocation decisions.

Bond pricing is not a single open problem. It is a lattice: a cleanup-cost lower bound (§7.5.1), a scope multiplier (§7.5.2), a reputation discount (§7.5.3), the Bonded Advisor mechanism (§7.5.7), and the competitive-insurance market (§7.5.8). Each is precise enough to disprove.

The infrastructure is running. The forest is building. The covenants are auditable. The sovereign is legible. The advisor is bondable. The market is open for bids.

Solutions to the exercises

7.1 *(exercise on page 45)* The rejected card violates *temporal* attenuation: Theorem 3.1 requires $C(\tau_k) \subseteq C(\tau_{k-1})$ along every dimension of the capability, and TTL is one such dimension—a child's expiry must not exceed its parent's. Here the child requests 20 minutes against a parent that expires at 15; even though the capability set {`fs:write:auth.ts`} is a subset of α's, the delegation as a whole is not a narrowing.

An acceptable card is {`fs:write:auth.ts`}, TTL $\leq$ 15 minutes—dropping `db:read` (a valid narrowing of the capability set) and matching or shortening the TTL. A strictly narrower example is {`fs:read:auth.ts`}, TTL = 10 minutes, if the deployment's capability lattice places `read` below `write` on the same resource.

7.2 *(exercise on page 46)* An advisory claim is a durable declaration that an agent intends to use a resource; it informs coordination (Definition 5.1's conflict graph) but does not make conflicting effects impossible. Every trace an enforced-lock regime allows, an advisory regime also allows—the lock regime is a special case that additionally forbids the conflicting trace—so advisory claims are strictly more permissive, never less.

That extra permissiveness is the feature Theorem 5.1 names: Sen's theorem shows that granting an agent a decisive personal domain (an enforced lock) can block a Pareto-superior team outcome the allocator cannot see coming. An advisory regime never grants that veto, so it never forecloses the outcome a lock would have blocked—at the cost of also admitting traces where two agents genuinely do collide. §5.3's appeals and preemption path is exactly the mechanism that prices the cost of that added permissiveness rather than eliminating it.

7.3 *(exercise on page 46)* First, reputation compensates nobody for the harm already done. A one-shot defector takes the gain from the first (and only) defection and leaves before any future exclusion can matter to it—exactly the "one-shot defectors" adversary class named above. Second, cheap identities let the actor whitewash the loss: register a fresh agent ID, and the reputation ledger has nothing to slash. A pre-funded bond avoids both failure modes because it creates recoverable value *before* authority is granted and keys the exposure to the durable principal (§4.4) rather than to the disposable agent identity—it prices and

contains harm at the moment of the first defection, not on the promise of a future one. What a bond does *not* do is prove the work will be correct; it only ensures that if the work is wrong, someone has already paid for the possibility.

7.4 *(exercise on page 46)* Fix A's choice and compare its two rows against each of B's columns. Against B:T, A gets 3 from T and 4 from F, so F is better ($4 > 3$). Against B:F, A gets 0 from T and 1 from F, so F is again better ($1 > 0$). F therefore strictly dominates T for A regardless of B's move; the game is symmetric, so the same holds for B. Checking the three remaining cells: at $(T, T) = (3, 3)$ either player can deviate to F and gain ($4 > 3$); at $(T, F) = (0, 4)$, A can deviate to F and gain ($1 > 0$); at $(F, T) = (4, 0)$, B can deviate to F and gain ($1 > 0$). Only $(F, F) = (1, 1)$ survives—neither player gains by unilaterally switching back to T ($0 < 1$)—so it is the unique Nash equilibrium, exactly as the table's caption states.

Provenance note. An earlier draft of this chapter's stage-game table used different payoffs that a review flagged as *not* a genuine Prisoner's Dilemma—mutual truth was not Pareto-dominant, and the proposed mutual-punishment strategy was not sequentially rational under that table. Table 5 above is the corrected table; this exercise verifies the correction that is already in the chapter, not the flawed table that review replaced.

7.5 *(exercise on page 46)* At $\delta = 0.34$: $\delta^2 = 0.1156$, $1 + \delta + \delta^2 = 1.4556$, $2\delta = 0.68$, and $0.68 \times 1.4556 = 0.989808$, so $f(0.34) = 0.989808 - 1 = -0.010192$ [verified]—negative. At $\delta = 0.35$: $\delta^2 = 0.1225$, $1 + \delta + \delta^2 = 1.4725$, $2\delta = 0.7$, and $0.7 \times 1.4725 = 1.03075$, so $f(0.35) = 1.03075 - 1 = 0.03075$ [verified]—positive. f is continuous and strictly increasing on $(0, 1)$ (every term's coefficient is positive), so a sign change from $\delta = 0.34$ to $\delta = 0.35$ proves a root lies strictly between them—the Intermediate Value Theorem, not a numerical coincidence.

This matches two independent committed artifacts. `proofs/economics/delta-threshold.z3`'s second query (`delta-threshold.expected.txt`) asserts the root lies in $[0.34, 0.35]$ and returns `sat`, agreeing with the sign change computed here. Independently, `proofs/economics/sweep-delta.sh`'s committed sample sweep (documented in `proofs/economics/README.md`) reports the model-checked IC invariant `VIOLATED` at $\delta = 0.34$ and `HOLDS` at $\delta = 0.35$—the same sign change, checked by a state machine instead of a closed-form inequality.

7.6 *(exercise on page 46)* The script asks, in order: (1) does a real root of $2\delta^3 + 2\delta^2 + 2\delta - 1 = 0$ exist in $[0, 1]$; (2) does that root lie in the tighter interval $[0.34, 0.35]$; (3) does a *second*, distinct root exist in $[0, 1]$ (a uniqueness check, expected to fail).

Pinned verdict sequence (`proofs/economics/delta-threshold.expected.txt`):

```
sat
(
  (define-fun delta () Real
    (root-obj (+ (* 2 (^ x 3)) (* 2 (^ x 2)) (* 2 x) (- 1)) 1))
)
sat
(
  (define-fun delta () Real
    (root-obj (+ (* 2 (^ x 3)) (* 2 (^ x 2)) (* 2 x) (- 1)) 1))
)
unsat
```

Reading the three lines against the three questions: `sat` (a root exists, printed in Z3's algebraic `root-obj` form rather than a decimal); `sat` again with the identical model (that same root also satisfies the tighter bound, so it lies in $[0.34, 0.35]$); `unsat` (no second, distinct δ_2 in $[0, 1]$ satisfies the cubic, so the root is unique). All three together are exactly Theorem 7.2's threshold, existence-and-uniqueness, mechanized.

7.7 *(exercise on page 46)* At $\delta = 0.30$: $\delta^2 = 0.09$, $1 + \delta + \delta^2 = 1.39$, $2\delta = 0.6$, and $0.6 \times 1.39 = 0.834$ [verified]. The inequality $1 < 0.834$ is false, so the deviation is profitable and the invariant `NoUnilateralDeviationPositive` fails—matching the table's stated verdict and the committed sweep row `0.30 VIOLATED` (`proofs/economics/README.md`). At $\delta = 0.35$ (`proofs/economics/claim_signaling.cfg`'s own default, `DeltaNum=35`, `DeltaDen=100`), $2\delta(1 + \delta + \delta^2) = 1.03075$ from the worked example above, so $1 < 1.03075$ holds and the invariant holds—matching `claim_signaling.cfg` being the one committed configuration and the sweep row `0.35 HOLDS`.

The trace: at $\delta = 0.30$, TLC's search reaches a state where an agent deviated to F once in round one, was punished for three rounds at the mutual-claim payoff, and reached `round = Horizon` with `deviated` true; because the deviator's discounted score from that trajectory (`actualScore`) exceeds the score it would have earned by cooperating throughout (`followScore`), the state violates `NoUnilateralDeviationPositive` and TLC reports it as a counterexample. No such state exists at $\delta = 0.35$: the same one-shot-deviation trajectory now scores strictly worse than cooperating, so every reachable state at `round=Horizon` satisfies `actualScore` $\leq$ `followScore` and the search completes with "No error has been found." The committed trace is `proofs/economics/claim_signaling_delta30.run.log`, produced by the negative-control configuration `claim_signaling_delta30.cfg`; the proofs workflow asserts on every run that it still violates.

7.8 *(exercise on page 46)* The committed sample output (`proofs/economics/README.md`) reports the invariant `VIOLATED` for every δ from 0.30 through 0.34 and `HOLDS` from 0.35 through 0.40, so the crossover—the smallest δ for which the invariant holds—is pinned at

```
crossover (smallest delta where invariant HOLDS) = 0.35
closed-form root (delta-threshold.z3)            = 0.3425
PASS: crossover matches closed-form within integer-grid rounding.
```

The script's own pass criterion (its exit code) is that the crossover lands in $\{0.34, 0.35\}$—the two integer-grid points bracketing the closed-form root $\delta^\star \approx 0.3425$ from Exercise 7.5. A crossover anywhere else would mean the model-checked state machine and the closed-form cubic disagree about where cooperation becomes sustainable, which is exactly the regression this script exists to catch on every PR touching `proofs/economics/` (§A).

7.9 *(exercise on page 46)* The attacker spawns many low-history Sybil insurer identities, each posting the mandated deposit B_{dep}; each identity underbids honest insurers by an arbitrarily small ε, wins the auction, collects premiums on every no-loss round, and defaults the moment a real loss lands. Because the slash is capped at $\min(B_{\text{dep}}, B_T)$, once $B_{\text{dep}} \geq B_T$ every dollar of deposit beyond the coverage ceiling is recovered intact on no-loss transactions—it never reaches the commons. Concretely, if concurrent aggregate exposure across the Sybil's identities is not reserved, one deposit can underwrite several simultaneous liabilities at once, multiplying the attack for free.

Correction to the naive fix: full reimbursement of a single loss does not, by itself, show the attacker profits—that also needs the premium/loss distribution, exposure reuse across identities, or an external benefit from sabotage. Raising B_{dep} alone does not close A5, because the slash cap still bounds the attacker's downside at B_T regardless of how large a deposit sits above it. The defense that actually closes it is the composite mechanism named in the hint: an unrecoverable per-identity onboarding cost C_{kyc} (so spawning K Sybils costs $K \cdot C_{\text{kyc}}$, not zero), reputation gating that caps a new identity to low-coverage transactions until it accumulates a track record, a per-risk-class B_{dep} tuned to saturate its own coverage cap exactly, and atomic reservation so aggregate outstanding exposure across an identity's positions never exceeds its posted capital. Independent audits and pairwise-concavity checks are the remaining defense against collusion among identities that individually stay under any single reservation limit.

7.10 *(exercise on page 46)* With $x = q_{\text{floor}} - \mu$, the chapter's own closed form gives $\pi_C = x/3$, $\pi_D \to x$ as $\varepsilon/x \to 0$, and $L = 5x$, so

$$p_d^\star = \frac{\pi_C - (1-\delta)\pi_D}{L + \delta\pi_D} = \frac{x/3 - (1-\delta)x}{5x + \delta x} = \frac{1/3 - (1-\delta)}{5 + \delta}$$

after cancelling the shared factor x. At $\delta = 0.99$: $1/3 - (1-0.99) = 1/3 - 0.01 = 97/300$, and $5 + 0.99 = 5.99 = 599/100$, so $p_d^\star = (97/300)/(599/100) = 9700/179700 = 97/1797 \approx 0.05398$ [verified]—about 5.4% per round, close to but distinct from the chapter's own worked point at $\delta = 0.95$ ($p_d^\star \approx 0.0478$): a more patient cartel (δ closer to 1) needs a slightly *higher* detection probability to stay fragile, because the discounted value of continuing the cartel is larger.

Cheap fresh identities shorten the effective horizon a punished member faces and let it shed the punishment entirely by re-registering—exactly the Sybil failure mode Exercise 7.3 names for reputation alone. A *principal-bound* bond deters re-entry only when the expected forfeiture plus the onboarding and reputation-maturation cost of a fresh identity exceeds the gain from re-entering the cartel; a bond that is reusable across identities, or sized below that threshold, merely *prices* re-entry rather than deterring it. The bond mechanism by itself never *prevents* re-entry—nothing in this chapter's architecture stops a new registration—so the honest answer is: prices always, deters conditionally, prevents never.

7.11 *(exercise on page 46)* This is a design obligation, not a closed theorem: no mechanism in this chapter proves a Pareto ordering for a posted-price catalog, so what follows is a defensible proposal, not a fictional derivation. A posted-price catalog quotes premiums by manifest risk class, principal history, exposure duration, and tail-capital band, recalibrated periodically against a hard reserve constraint—no per-transaction bidding, no auction latency. It conditionally improves on the static escrow of §7.5.2 exactly when the posted quote sits below the principal's idle-capital cost of self-insuring via a static bond, while the insurer remains individually rational and its reserves still cover the same tail risk the competitive market would have priced. Relative to §7.5.8's bid-based mechanism, a posted catalog trades away instance-specific price discovery (Rothschild–Stiglitz's $q_I \to \mathbb{E}[d]$ convergence under competition) for lower manipulation surface and no per-transaction latency—the same trade the A6 cartel analysis makes salient in the other direction, since a catalog has no bid pattern for a colluding cartel to distort in the first place.

The proposal depends most on *calibrated loss/tail dependence*: the catalog's bands must actually track the loss distribution within each class, and its capital reserve must be enforceably adequate against the tail, or the "conditional" improvement collapses into either under-pricing risk (insurer insolvency) or over-pricing it (pricing capable agents out, the same failure the threat-class bands of §7.5.4 already warn against). The CC/NC/RP/NS assumptions of §7.5.8 would need explicit class-conditional definitions before any formal Pareto claim could be stated for this alternative—this exercise does not supply that derivation, and neither should its answer key pretend to.

7.12 *(exercise on page 47)* The committed log (`proofs/bonded/conservation/Conservation.run.log`) reports:

```
Model checking completed. No error has been found.
26818 states generated, 1716 distinct states found, 0 states left on
    queue.
The depth of the complete state graph search is 9.
```

These are two different numbers, and the difference is the exercise. TLC *generates* a state every time it evaluates a transition, including transitions that land back on a state it has already seen by a different path—26,818 counts every such evaluation. The *reachable* state count is the number of **distinct** states TLC actually found, 1,716: every other generated state was

a re-visit, not a new point in the state space. (Table 9 now carries both figures for this row; both are pinned from the same committed log, and only 1,716 answers "how many reachable states are there.")

The invariant named in `Conservation.cfg`'s `INVARIANTS` block that gives the module its name is `Conservation`, defined as `TotalFree + TotalEscrow + burned = minted`—the free-form statement of Theorem 10.1 at the granularity of individual mint/stake/slash/refund/payout operations rather than the three-bucket accounting state. The same run also checks `NoNegative` (no balance ever goes negative) and `Safety` (their conjunction); the log's "No error has been found" covers all three across every one of the 1,716 reachable states, to search depth 9.

7.13 *(exercise on page 47)* No run is needed to predict this: it follows from the definitions already in the spec. `Conservation` asserts $\mathit{TotalFree} + \mathit{TotalEscrow} + \mathit{burned} = \mathit{minted}$. Immediately after `Init` (all four quantities zero, satisfying the invariant trivially), the very first enabled `Mint`(a, amt) step increases *TotalFree* by *amt* while leaving *minted* at 0 under the stated weakening, so the left side becomes $\mathit{amt} > 0$ while the right side stays 0. `Conservation` fails at depth 1—the shallowest possible counterexample, one step past the initial state, with no need to explore further.

`NoNegative`, by contrast, does *not* catch this: every quantity involved (`free`, `escrow`, `burned`, `minted`) stays non-negative under the weakened `Mint`—the bug is a bookkeeping omission, not a sign violation, and `NoNegative`'s four non-negativity clauses are silent about the relationship *between* the buckets. `Safety`, defined as `Conservation` $\wedge$ `NoNegative`, fails only because its `Conservation` conjunct does; this is exactly why the module checks `Conservation` as its own named invariant rather than folding it silently into `Safety` alone; a reader who only watched `NoNegative` would see nothing wrong.

7.14 *(exercise on page 47)* Model a fresh token t, an issuance event Issued(u, t) tied to the user's mailbox, storage of an unguessable token (or a protected hash of it) with an expiry and an unused flag, delivery over a mailbox channel the attacker can read but not write into unprompted, redemption over an attacker-controlled network under *The Anchor Protocol*'s Dolev-Yao assumptions, an atomic consume step, and recovery-key or session rotation on successful redemption.

The secrecy property is that the token (and any recovery secret it unlocks) stays unknown to the attacker absent mailbox compromise. The authentication property is injective: Consumed$(u, t) \Rightarrow$ inj-event(Issued(u, t)), exactly the form §12 already states for magic-link single-use—every consumption traces back to one, and only one, issuance, which is what rules out the race window between two concurrent redeemers. Add a concurrent-replay capability (two redemption attempts racing on the same token) and a mailbox-compromise capability (the attacker reads, but does not yet control, the delivery channel), neither of which the Phase 1 Harbor Card model needs: that model's attacker forges signatures or replays chains, but it has no notion of a *delivery channel* an attacker might passively observe, because Harbor Cards are minted and verified entirely within the daemon's own boundary. Time and expiry also need an explicit abstraction here—ProVerif has no wall-clock semantics on its own—whereas the Harbor Card model's TTL is already folded into the token's signed payload.

7.15 *(exercise on page 47)* This is an open mechanization obligation, not a solved one: Demers et al. is the epidemic-replication paper the chapter leans on, but no numbered theorem in it is stated in a form that yields this protocol's $\Theta(\log m)$ all-informed bound by direct substitution—the paper's push/pull rumor-spreading results assume a complete, uniform-peer overlay with reliable rounds, which is an assumption this chapter states but does not itself formalize. The mechanization obligation is therefore two steps, not one: first state the push/pull rumor-spreading theorem precisely, under exactly the complete-graph, independent-peer, reliable-round assumptions this chapter already uses (§7.4's "observable history" bullet leans on the same overlay); only then port that precise statement to a proof

assistant. Citing the paper is not, by itself, a proof object.

On a general connected graph the $\Theta(\log m)$ bound does not transfer as stated: convergence time depends on the graph's conductance or mixing time, not on m alone, and the honest replacement bound has a term roughly inverse in conductance, plus a tail probability rather than a clean deterministic count. A disconnected graph is the degenerate case that breaks the claim outright: gossip never converges across components that share no edge, no matter how long the protocol runs, because there is no path for a rumor to cross. A connected-but-bottlenecked mesh sits between the two: dissemination across the bottleneck cut is throttled by the cut's capacity, so a mechanized version of this bound would need to separate *safety* (no revocation is ever falsely accepted or falsely dropped, which holds regardless of connectivity) from *liveness* (every revocation eventually reaches every device, which needs the connectivity and mixing assumptions this section has so far only asserted in prose).

7.16 *(exercise on page 47)* An attacker with no special privilege submits a large volume of authenticated-looking but distinct revocation IDs—each one legitimate on its own, since the filter cannot distinguish a real revocation from a manufactured one by content alone. Each insertion fills buckets and, past a certain load, drives kick-chain failures in the filter's relocation step. Past the load at which Fan et al.'s false-positive bound holds, the filter is pushed outside the regime the bound was proved for: insertions the filter can no longer place are either rejected or force evictions of *existing* entries, so a legitimate revocation already in the filter can be silently displaced.

The invariant that breaks is completeness of revocation—every previously revoked credential should still test positive—not the false-positive bound itself, which stays a probabilistic guarantee about *non*-members. The 0.95 ceiling defeats the attack described above only in the sense of preserving the regime the bound was proved in: refusing further insertions past that load keeps every already-inserted revocation intact, at the cost of refusing new ones (which is a capacity failure, loud and observable, rather than a silent integrity failure). It does not defeat a determined pollution attempt on its own, because an attacker willing to keep submitting IDs simply keeps the filter permanently at its ceiling, denying legitimate revocations a slot. Closing that requires what the ceiling alone does not provide: authenticated revocation provenance (only a party who can prove standing may insert), per-principal insertion quotas, an exact overflow log for anything the filter had to refuse, and a fail-closed policy—if insertion fails, the caller must not proceed as if the revocation succeeded. A probabilistic filter must never be allowed to silently drop an authoritative revocation.

References

[1] Thomas Hobbes. *Leviathan, or The Matter, Forme and Power of a Common-Wealth Ecclesiasticall and Civill*. Andrew Crooke, London, 1651. ↑ Cited on pages 6 and 40.

[2] John Locke. *Two Treatises of Government*. Awnsham Churchill, London, 1689. ↑ Cited on page 40.

[3] Amartya Sen. The Impossibility of a Paretian Liberal. *Journal of Political Economy*, 78(1):152–157, 1970. https://doi.org/10.1086/259614 ↑ Cited on pages 16 and 40.

[4] Andrew J. I. Jones and Marek Sergot. On the Characterisation of Law and Computer Systems: The Normative Systems Perspective. In *Deontic Logic in Computer Science: Normative System Specification*, pages 275–307. John Wiley and Sons, 1993. ↑ Cited on page 13.

[5] Robert J. Aumann. Subjectivity and Correlation in Randomized Strategies. *Journal of Mathematical Economics*, 1(1):67–96, 1974. https://doi.org/10.1016/0304-4068(74)90037-8 ↑ Cited on pages 1, 7, and 10.

[6] Hector Garcia-Molina and Kenneth Salem. Sagas. In *ACM SIGMOD International Conference on Management of Data*, pages 249–259, 1987. ↑ Cited on page 40.

[7] Jim Gray and Andreas Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1993. ↑ Cited on page 19.

[8] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992. ↑ Cited on page 19.

[9] Anand S. Rao and Michael P. Georgeff. Modeling Rational Agents within a BDI-Architecture. In *International Conference on Principles of Knowledge Representation and Reasoning (KR)*, pages 473–484, 1991. ↑ Cited on page 41.

[10] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. In *ACM Symposium on User Interface Software and Technology (UIST)*, 2023. ↑ Cited on page 41.

[11] Erich Owens. The Anchor Protocol: A Mechanically Analyzed Control Plane for Local Agent Swarms. Curiositech LLC Technical White Paper, May 2026. ↑ Cited on pages 7, 11, 11, 12, 13, 23, 23, 24, and 41.

[12] Leslie Lamport. *Specifying Systems: The TLA$^+$ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. ↑ Cited on page 36.

[13] Leslie Lamport. The Part-Time Parliament. *ACM Transactions on Computer Systems*, 16(2):133–169, 1998. ↑ Cited on page 45.

[14] Diego Ongaro and John Ousterhout. In Search of an Understandable Consensus Algorithm. In *USENIX Annual Technical Conference (ATC)*, pages 305–320, 2014. ↑ Cited on page 45.

[15] Jack B. Dennis and Earl C. Van Horn. Programming Semantics for Multiprogrammed Computations. *Communications of the ACM*, 9(3):143–155, 1966. ↑ Cited on pages 7 and 41.

[16] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014. `https://doi.org/10.14722/ndss.2014.23212` ↑ Cited on page 41.

[17] William Vickrey. Counterspeculation, Auctions, and Competitive Sealed Tenders. *The Journal of Finance*, 16(1):8–37, 1961. ↑ Cited on page 41.

[18] Roger B. Myerson. Optimal Auction Design. *Mathematics of Operations Research*, 6(1):58–73, 1981. ↑ Cited on page 41.

[19] Jonathan Hickman. *House of X / Powers of X*. Marvel Comics, 2019. ↑ Cited on page 19.

[20] Bruno Blanchet. Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1-2):1–135, 2016. ↑ Cited on page 13.

[21] Michael Rothschild and Joseph Stiglitz. Equilibrium in Competitive Insurance Markets: An Essay on the Economics of Imperfect Information. *Quarterly Journal of Economics*, 90(4):629–649, 1976. ↑ Cited on pages 4 and 30.

[22] Charles Wilson. A Model of Insurance Markets with Incomplete Information. *Journal of Economic Theory*, 16(2):167–207, 1977. ↑ Cited on pages 4 and 30.

[23] Guy Theraulaz and Eric Bonabeau. A Brief History of Stigmergy. *Artificial Life*, 5(2):97–116, 1999. ↑ Cited on page 15.

[24] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate Transparency. IETF RFC 6962, June 2013. ↑ Cited on pages 14 and 43.

[25] Bin Fan, David G. Andersen, Michael Kaminsky, and Michael D. Mitzenmacher. Cuckoo Filter: Practically Better Than Bloom. In *ACM CoNEXT*, 2014. ↑ Cited on page 47.

[26] Alan Demers, Dan Greene, Carl Hauser, Wes Irish, John Larson, Scott Shenker, Howard Sturgis, Dan Swinehart, and Doug Terry. Epidemic Algorithms for Replicated Database Maintenance. In *ACM Symposium on Principles of Distributed Computing (PODC)*, 1987. ↑ Cited on page 47.

[27] Adi Shamir. How to Share a Secret. *Communications of the ACM*, 22(11):612–613, 1979. ↑ Cited on page 44.

[28] Elinor Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press, 1990. ↑ Cited on pages 8 and 40.

[29] Vitalik Buterin and Virgil Griffith. Casper the Friendly Finality Gadget. arXiv:1710.09437, 2017. ↑ Cited on page 7.

[30] Ethan Buchman. Tendermint: Byzantine Fault Tolerance in the Age of Blockchains. M.A.Sc. thesis, University of Guelph, 2016. ↑ Cited on page 7.

[31] Ethereum Foundation. Ethereum 2.0 Phase 0 – The Beacon Chain Specification. `https://github.com/ethereum/consensus-specs`, 2020. ↑ Cited on page 7.

[32] Mark S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. Ph.D. dissertation, Johns Hopkins University, 2006. ↑ Cited on page 7.

[33] Carl M. Ellison, Bill Frantz, Butler Lampson, Ron Rivest, Brian M. Thomas, and Tatu Ylonen. SPKI Certificate Theory. IETF RFC 2693, 1999. ↑ Cited on page 7.

[34] Kevin Werbach. *The Blockchain and the New Architecture of Trust*. MIT Press, 2018. ↑ Cited on page 8.

[35] Vitalik Buterin. A Next-Generation Smart Contract and Decentralized Application Platform. Ethereum white paper, 2014. ↑ Cited on page 8.

[36] Universal Commerce Protocol. Specification and reference implementation (Apache-2.0), Google, Shopify, et al., 2026. `https://ucp.dev`. ↑ Cited on page 41.

[37] Agent Payments Protocol (AP2): Mandates. Verifiable-credential authorization chain: SD-JWT+kb checkout mandate, JWS detached-content merchant authorization, JCS canonicalization, 2026. `https://ap2-protocol.org`. ↑ Cited on page 41.

[38] Agentic Commerce Protocol. OpenAI and Stripe, 2025. `https://agenticcommerce.dev`. ↑ Cited on page 41.

[39] DataDome. Agent Trust Management and the Universal Commerce Protocol, 2026. `datadome.co/agent-trust-management/universal-commerce-protocol/`. ↑ Cited on page 41.

A Formal Verification Status

Artifact availability. Every artifact named in Table 9 is bundled at `https://portdaddy.dev/whitepaper/artifacts/`. Runtime components are deployed inside the Port Daddy daemon binary and exposed through its public APIs; the source for the verified Rust core is the open-source `harbor-card-rs` crate (accompanying chapters, §5).

Table 9: Verification status of critical claims. "Method" is the formal technique; "Result" is its outcome; "Artifact" names the public bundle file. A claim with both a formal artifact *and* a runtime conformance test is fully closed (**closed**); a claim with formal model but no deployed runtime is spec-only (*spec-only*); a claim with empirical evidence but no formal mechanization is partial (PARTIAL).

Claim	Method	Result	Artifact
Coordination channel isolation (§4.2)	ProVerif (Dolev-Yao)	**closed** 3 properties TRUE	`isolation.pv`
Passkey device pairing	ProVerif	**closed** 3 properties TRUE	`passkey-pair.pv`
Federated security (§12)	ProVerif, 3-of-4 compromise	**closed** no attacker reach	`federated.pv`
Conservation Theorem (§10.3)	TLA+ bounded TLC	**closed** 1,716 reachable states (26,818 generated), invariant holds	`Conservation.tla`
No-overdraft lemma (§7)	fast-check property test	**closed** 4 invariants, randomized ops	bonds-property tests
Algorithm-confusion immunity	ProVerif (pinned vs. naive)	**closed** pinned TRUE; counter-trace produced	`algconfusion.pv`
Delegation chain replay	ProVerif at depth 3	**closed** chain authenticity TRUE	`chain-replay.pv`
Magic-link single-use (§12)	ProVerif (private-channel cap)	**closed** injective-event TRUE; 7/7 runtime conformance	`magic-link.pv`
Cuckoo-filter pollution resistance	fast-check, 5 invariants	**closed** FP rate within $2B/2^f$ at load 0.95	cuckoo property tests
Merkle-Forest binding (§4.2)	EasyCrypt + fast-check	PARTIAL skeleton discharged; PROM-formal version not in scope	`binding.ec`
Auction/static comparison (§7.5.8)	Monte Carlo, 36 configs × 2k trials	PARTIAL finite-grid result; not a universal Pareto theorem	`simulation.mjs`
Sybil A5 (§7.5.9)	Monte Carlo over K Sybils	PARTIAL coverage-bounded ceiling at B_T	`simulation-sybil.mjs`
Cartel repeated-game A6	Monte Carlo + expected-value recurrence	PARTIAL $p_d^\star \approx 0.0478$ at $\delta=0.95$ in supplied model	`simulation-cartel.mjs`
Claim-signaling IC (§7)	TLA+ (TLC) + Z3 SMT	**closed** unique root $\delta^\star \approx 0.3425$; IC invariant fails at $\delta = 0.30$, holds at $\delta = 0.35$	`claim_signaling.tla`, `delta-threshold.z3`
Threat-band defensibility (§7.5)	Monte Carlo, 7×3 grid	**closed** matched bands absorb target class within 1.10× upper bound	`threat-bands.mjs`
Passkey pairing runtime	—	*spec-only* model exists; no runtime	`passkey-pair.pv`

Federated KMS Worker	Cloudflare —	*spec-only* model exists; no Worker yet	`federated.pv`

Limitations. Four mechanizations are out of scope for this paper but explicitly named so a reader can audit what is and is not proved: a PROM-formal Merkle binding in EasyCrypt; a Coq or Lean mechanization of the Pareto strategic game; a formal analysis of the composite Sybil-deterrence mechanism (C_{kyc} + reputation gating + per-class B_{dep}); and a multi-class generalization of the repeated-game cartel result to heterogeneous coordination classes (§8). The present claims stand on the evidence stated in Table 9.

B Worked Example: One Agent, All Layers

To make the three-layer architecture concrete, this appendix traces one agent—call it α—through a single bounded task: *"Fix the login bug in* `auth.ts` *and regression-test it."* The example exercises capability issuance, advisory claims, the evidence chain, a mid-task crash, successor resumption, and final settlement. All values are illustrative rather than tuned.

The architecture so far has been described layer by layer. The point of this appendix is to watch the layers *compose*: how a single bounded task progresses through the system, where each layer earns its keep, and what the daemon does *not* touch. The example is deliberately mundane—one small bug, one agent, one principal—because the value of the architecture should be legible in the small before it is trusted in the large.

The task. A principal P wants an agent α to fix a login bug in a hypothetical file `auth.ts` and regression-test it. The acceptance criteria are stated up front:

1. the failing test `login.regression.test.ts` passes;
2. no other test regresses;
3. the diff touches only `auth.ts` and the test file.

P estimates the cleanup cost of α's defection or sabotage at \$5—one operator-hour at \$5/hr equivalent, since this is a local agent rather than a cloud workload. Figure 9 traces what follows across the three architectural layers.

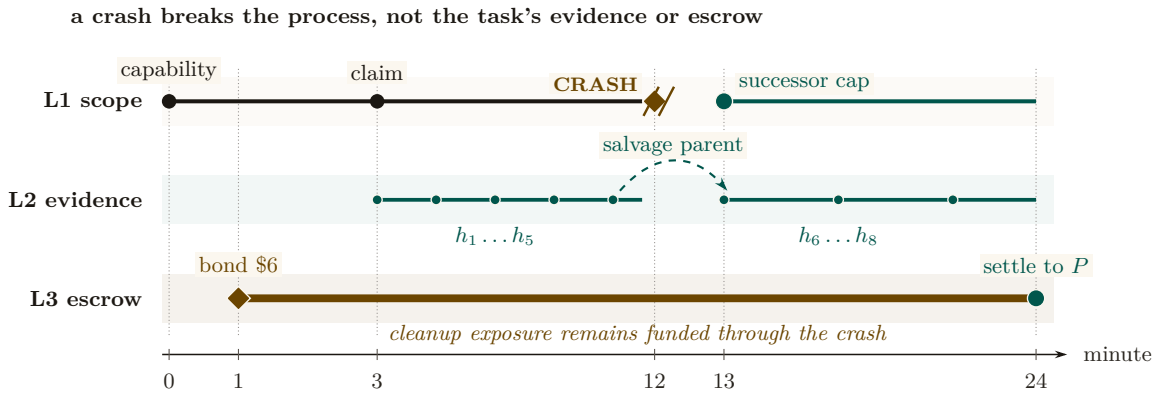


Figure 9: The worked example is a continuity braid over one task. Structural authority belongs to a process and visibly breaks at minute 12; the successor receives a new capability at minute 13. The evidence chain does not break: its salvage-parent edge binds $h_1 \dots h_5$ to $h_6 \dots h_8$. The escrow rail spans the entire task and settles at minute 24. Identity is disposable, while attribution and pre-funded cleanup exposure persist.

Phase A: Setup (minutes 0–1)

Before α writes a single byte, two artifacts must exist: a capability that bounds what α can touch, and a bond that prices what α might break.

Step 1: Capability issuance (Layer 1). P requests a Harbor Card for α via the daemon, specifying the capability set {`fs:read:repo`, `fs:write:auth.ts`, `fs:write:tests/**`, `cmd:test`} and a TTL of 30 minutes. The Anchor Protocol (§3, accompanying chapters) signs the card. The effect is structural rather than advisory: α can prove its identity and capabilities to any verifier, but it *physically* cannot write to, e.g., `database.ts`—attempts return a capability error before the file system is even touched. This is the "physically" that makes the Layer 1 promise non-negotiable.

Step 2: Bond posting (Layer 3). With the capability in place, the Bonded Advisor (§7.5) quotes a bond proportional to the cleanup cost: $B_\alpha = \$5 \cdot (1 + s) = \$6$ at slope $s = 0.2$. P posts \$6 into escrow. The settlement preconditions are now fixed up front: (1) acceptance criteria met $\Rightarrow$ \$6 returned, (2) partial completion $\Rightarrow$ pro-rata, (3) sabotage $\Rightarrow$ \$6 liquidated to the commons treasury. The bond is the conversion of α's "character" into a settled amount; once it is posted, the question of whether α is trustworthy no longer determines settlement.

Phase B: Work begins (minutes 1–12)

Phase A's contracts are now in force. α does the work the principal hired it for, and the daemon records what happens.

Step 3: Advisory claim (§5). α claims `auth.ts` via `pd session files add auth.ts`. The daemon's conflict graph is empty for this path, so the claim succeeds with no conflict notification; had another agent already held the path, α would have been told and would have decided locally whether to coordinate or wait. The claim's TTL is bounded by the Harbor Card's—30 minutes—so a crashed α cannot squat on the file indefinitely.

Step 4: Evidence chain begins (Layer 2). α opens the work with a scope note: *"Scope: `auth.ts`. Hypothesis: cookie-domain handling on subdomain login fails. Validation: `npm test login.regression`."* The note's content hash h_1 chains into the session's Merkle root $R^{(1)}_{\text{session}}$, and from then on every action α takes—a read receipt for `auth.ts`, a diff blob for the partial fix, the stdout of the failing test run—is appended as a new entry. Five notes deep, $R^{(5)}_{\text{session}}$ is the canonical commitment to α's state of work. The chain is the part of the architecture that survives anything that happens next.

Phase C: Crash and recovery (minutes 12–13)

The crash is where the architecture stops being an abstraction. In Phase B everything went right; in Phase C something goes wrong, and the layers either earn their keep or they don't.

Step 5: Mid-task crash (§6). At minute 12, α's process crashes—let's say the LLM provider timed out. The daemon's heartbeat monitor observes α as unresponsive after 90 seconds and marks the session abandoned. α's claim on `auth.ts` releases on TTL; the evidence chain is sealed at $R^{(5)}_{\text{session}}$ and committed into the harbor root $R_{\text{harbor},\ell}$ at the next epoch boundary. *Critically, no information is lost*: the partial diff, the test output, and α's scope note are all in the chain. The crash terminated a process; it did not terminate the work.

Step 6: Successor agent. P spawns a fresh agent β with a new Harbor Card and the same capability set. β's first action is `pd salvage`: the daemon returns the abandoned session, the scope note, the partial diff, and the last test output. β reads the scope note ("cookie-domain handling on subdomain login"), inspects the partial fix that α left behind, and finishes the work. The new evidence chain $R^{(\beta)}_{\text{session}}$ explicitly references $R^{(5)}_{\text{session}}$ as its salvage parent. Attribution is preserved across the identity change (§4.4)—the diff that ships at minute 24 is traceable back to the same principal P that posted the bond at minute 1, even though the agent that did most of the work is gone.

Phase D: Settlement (minute 24)

The work is done. The daemon now decides what was actually accomplished and acts on the settlement preconditions fixed in Phase A.

Step 7: Settlement. β's final action is `pd done`. The daemon's acceptance-criteria checker runs the regression suite, confirms `login.regression.test.ts` passes, confirms no other test regresses, and confirms the diff is bounded to the two files declared in the Harbor Card. All three criteria met. The daemon releases $6 from escrow to P, signs a settlement record over the evidence-chain root, and appends that record to the harbor Merkle forest (§4.2). The settlement is now itself an immutable, attestable artifact.

What did each layer do?

- *Layer 1 (structural)* kept the agent inside the declared blast radius. The crash, the recovery, and the settlement all happened inside a capability-bounded scope. A capability error at minute 7 would have been a fast-fail, not a corrupted repository.
- *Layer 2 (attribution)* kept the work durable across the crash. Without the Merkle-chained evidence, β would have started from zero; with it, β resumed at minute 12's state.
- *Layer 3 (economic)* kept the principal honest in scoping. P posted real collateral and recovered it on completion; had α (or β) sabotaged, the cleanup cost would have been pre-funded.

What did *not* happen. The daemon never decided which agent should hold the file, never decided whether the fix was "good enough" in a subjective sense, and never punished the crash. It enforced what *cannot* be (Layer 1), recorded what *did* happen (Layer 2), and settled against the published criteria (Layer 3). The substantive judgments—"is this scope right", "is this fix correct"—remained with P and the acceptance-criteria checker. This is the Sen-grounded posture of §5 made operational.

C Scope Boundary Checklist

Section 12 (*Discussion and Limitations*) and Appendix A's own limitations paragraph carry the substantive boundary discussion; this closing checklist exists so a reader who jumped straight to the appendices does not have to hunt through the body for the three bounding assumptions that recur most.

- **Threat model exclusions:** Collusion across all independent organs (e.g., the logging daemon, the arbitrator, and the primary execution runtime) is assumed out-of-scope; if all enforcing components are compromised, the guarantees degrade to advisory.
- **Performance trade-offs:** Cryptographic assertions and WAL-checkpointing for per-write durability incur measurable I/O overhead. These mechanisms are designed for high-stakes coordination, not for bulk data processing or high-frequency trading.

- **Implementation maturity:** While the core protocols (Anchor, SQLite single-writer) are IMPLEMENTED and running in production, surrounding ecosystem components (like the decentralized judge markets or fully federated multi-sig routing) remain in PARTIAL or DESIGNED phases.

None of the three is a flaw the chapter is unaware of; each is a structural reality of building zero-trust coordination on top of POSIX primitives, already priced into the hedged claims made where each mechanism was introduced.